TECHNICAL PAPER

NeuroGraph
Adaptive Neural Gossip Protocol

neurograph_v4.3.1

Neurograph Research
2026

NeuroGraph — Adaptive Neural Gossip Protocol — Technical Paper (v4.3.1)

Abstract. This paper presents the complete technical specification of NeuroGraph ANGP v4.3.1, a DAG-based distributed ledger protocol that achieves consensus through emergent neural computation rather than traditional BFT agreement or Proof of Stake mechanisms. While NeuroGraph shares the goal of maintaining a consistent, append-only distributed ledger with blockchain systems, its consensus mechanism is fundamentally different: it replaces deterministic agreement protocols with emergent neural computation, making the "blockchain" label a useful comparison point but not a precise descriptor of the architecture. The system introduces a novel nine-layer architecture—Transaction, Prediction, Consensus, Reputation, Security, Network, Sharding, Slot, and Finality—where an Adaptive Directed Acyclic Graph with Hebbian learning serves as the central cognitive engine of each node. We formalize the Hebbian weight update rule with adaptive learning rate α, the dual-EMA reputation engine with 13 detection mechanisms (complemented by 2 attack detectors in Layer 5, totaling 15 across both layers), the cluster-aware weighted median consensus, and the VRF-based multi-proposer selection protocol. The security model is validated against 37 distinct attacker types (T0–T36) across 2664 nodes (333 shards $\times$ 8 nodes/shard), demonstrating robust resistance to coordinated attacks, Sybil clusters, slow poisoning, and adaptive adversaries. We provide formal definitions for the Three-Lane transaction processing architecture, the FCFS slot admission with anti-whale PoW, and the cross-shard Lock-Commit protocol with automatic refund on expiry.

Contents

1 Introduction

NeuroGraph ANGP (Adaptive Neural Gossip Protocol) v4.3.1 represents a fundamentally new approach to distributed consensus. Unlike classical blockchain and distributed ledger systems that rely on BFT agreement protocols (PBFT[1], HotStuff[2]), Proof of Work (Bitcoin[3]), or Proof of Stake (Ethereum 2.0[4], Algorand[5]), NeuroGraph achieves consensus through *emergent neural computation*: each node maintains an Adaptive Directed Acyclic Graph (AdaptiveDag) with Hebbian-inspired learning[6] that functions as a local neural network, and the global consensus emerges as the reputation-weighted median of all neural predictions.

The core insight is that honest nodes, when equipped with Hebbian learning, naturally converge toward consistent predictions because they observe the same underlying reality. In simulation, this shared reality is modelled by a synthetic validation signal $s(t) = A\sin(\omega t + \phi) + \mathcal{N}(0, \sigma^2)$, chosen for its well-characterised statistical properties (periodicity, noise, and observability) that exercise the full prediction pipeline. This signal validates the emergent-consensus dynamics but does not constitute a claim that general distributed consensus reduces to sinusoidal observation; real-world deployments would use application-specific prediction targets (e.g., transaction validity, block hashes, market data). Attackers, who must deviate from the shared signal to influence the consensus, are detected through their accumulated prediction errors and penalized via a multi-signal reputation engine. This creates a self-reinforcing feedback loop: honest nodes accumulate high reputation $\rightarrow$ their predictions dominate the weighted median $\rightarrow$ attackers' predictions diverge further $\rightarrow$ their reputation drops $\rightarrow$ their influence diminishes. This feedback loop is bootstrapped by the initial reputation neutrality: all nodes begin with $r = 0.5$ and a 150-step bootstrap period during which the reputation floor prevents premature penalization, allowing the honest majority to establish consensus before attacker influence can compound.

The system is implemented in Rust (edition 2021) with the following key properties:

- **333 shards** with 8 nodes per shard = 2664 active nodes
- **37 attacker types** (T0–T36): 16 base, 6 extended, 15 advanced
- **4-dimensional prediction space** (dim = 4) with SHA-512/256 hashing: signal amplitude, frequency, phase, and noise variance
- **Three-Lane Architecture** (Fast/Medium/Slow) for transaction classification
- **FCFS Slot Admission** with progressive PoW difficulty (anti-whale)
- **VRF-based proposer selection** with reputation weighting
- **Cross-shard Lock-Commit protocol** with automatic refund after 1000 steps

This paper is organized as follows: Section 1.1 motivates NeuroGraph's existence by examining the Blockchain Trilemma and its limitations in existing systems. Sections 2–10 detail each of the nine architectural layers. Section 11 presents the unified anomaly detection system. Section 12 analyzes the security properties under the 37-attacker model. Section 13 discusses performance characteristics. Section 14 presents simulation results including security validation, detection metrics, throughput, and profiling. Section 15 concludes.

[1] Castro & Liskov, "Practical Byzantine Fault Tolerance," *OSDI*, 1999 [4].
[2] Yin et al., "HotStuff: BFT Consensus with Linearity and Responsiveness," *ACM CCS*, 2019 [5].
[3] Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 [1].
[4] Buterin & Griffith, "Casper the Friendly Finality Gadget," 2017 [3].
[5] Gilad et al., "Algorand: Scaling Byzantine Agreements for Cryptocurrencies," *ACM SIGSAC*, 2017 [6].
[6] Hebb, *The Organization of Behavior*, Wiley, 1949 [7].

1.1 Motivation: The Blockchain Trilemma and the Case for Emergent Consensus

A widely discussed constraint in distributed ledger design is the so-called *Blockchain Trilemma*[1]: the difficulty of simultaneously maximizing **decentralization** (open, permissionless participation without trust assumptions), **security** (consistency and safety under adversarial conditions), and **scalability** (high throughput and low latency at network scale). Existing architectures typically make explicit or implicit trade-offs among these three dimensions, and the resulting compromises define the landscape of deployed protocols.

Sacrificing Scalability. Bitcoin[2] and classical Proof of Work chains achieve decentralization and security but sacrifice scalability: Bitcoin processes approximately 7 transactions per second (TPS) globally, limited by its 1 MB block size and 10-minute block interval. The bottleneck is structural—larger blocks or faster intervals require proportionally more bandwidth and storage, centralizing the set of nodes capable of validating the full chain. Ethereum 1.0's 15 TPS suffers from the same fundamental constraint[3]. These systems demonstrate that Nakamoto-style longest-chain consensus, while elegant in its simplicity, encodes a throughput ceiling that cannot be raised without degrading the decentralization property that makes the chain valuable.

[4]

of History timestamping mechanism, reaching peak throughputs of 60,000+ TPS in controlled benchmarks. However, this performance requires hardware specifications (128 GB RAM, 12-core CPU, high-bandwidth NVMe storage) [19] exclude the vast majority of potential validators, effectively concentrating validation capacity among operators able to provision high-end hardware and network infrastructure. Cosmos[5] and Polkadot[6] take a different route to the same destination: their hub-and-spoke and relay-chain architectures use bounded validator/committee sets and specialized inter-chain security mechanisms, introducing additional trust and coordination assumptions compared with unrestricted node-level participation. These systems illustrate a recurring pattern: high throughput and strong safety are achievable, but only by restricting the validator set to a bounded, identifiable group.

Sacrificing Security / Trustlessness. Some DAG-based systems—notably early IOTA[7]—attempted to achieve both decentralization and scalability by replacing the linear chain with a directed acyclic graph, allowing parallel transaction attachment without global ordering. However, IOTA's original Coordinator node (a centralized signing oracle that periodically checkpointed the DAG) constituted a fundamental security compromise: the system was neither decentralized nor trustless during its bootstrapping phase. The Coordinator was removed only in 2021 after years of development, illustrating the difficulty of achieving DAG-based consensus without a trusted fallback.

Restricting Permissionless Participation. Hashgraph[8] achieves fairness and asymptotically optimal communication complexity through gossip-about-gossip, but its reliance on a

[1] The term was popularized by Buterin (2014) and formalizes a trade-off observed across many systems; see also the scalability analysis in Croman et al., "On Scaling Decentralized Blockchains," *Financial Cryptography*, 2016 [17].

[2] Nakamoto, 2008 [1].

[3] Buterin, "Ethereum White Paper," 2014 [2].

[4] Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," 2018 [8].

[5] Kwon, "Tendermint: Consensus without Mining," 2014 [9].

[6] Wood, "PolkaDot: Vision for a Heterogeneous Multi-Chain Framework," 2016 [10].

[7] Popov, "The Tangle," 2018 [11].

[8] Baird, "The Swirlds Hashgraph Consensus Algorithm," 2016 [12].

known-member set for virtual voting restricts its applicability as an open, permissionless protocol, and its patent-encumbered status further limits adoption in the permissionless setting. These examples illustrate that achieving both DAG-based parallelism and permissionless openness remains an unsolved challenge.

The BFT Messaging Bottleneck. Classical all-to-all BFT protocols such as PBFT [4] exhibit $O(n^2)$ message complexity per consensus round to ensure safety under $f < n/3$ Byzantine faults. This quadratic cost caps practical committee sizes at ∼100–300 validators: beyond this, the messaging overhead dominates throughput and latency. Later protocols such as HotStuff [5] reduce the communication pattern to $O(n)$ per round via pipelined linear communication, but still rely on bounded voting committees and repeated rounds of authenticated communication. Algorand [6] addresses the committee-size constraint via cryptographic sortition (selecting a random sub-committee via VRF[1]), reducing the number of participants involved in each consensus instance, while introducing probabilistic guarantees regarding committee composition and therefore requiring explicit failure-probability parameters. Early Ethereum 2.0 sharding designs [18] proposed 64 shards each secured by ∼150 validators, with cross-shard communication requiring a latency-multiplied beacon-chain finality, limiting the aggregate throughput gain to a factor of $O(\text{shard\_count})$ with an additive cross-shard overhead; subsequent design iterations have further evolved this architecture.

Why NeuroGraph Exists. The preceding analysis reveals a common structural root of the Trilemma: *all existing approaches achieve agreement through explicit message exchange—either economic competition (PoW), stake-weighted voting (PoS), or deterministic protocol rounds (BFT).* The message complexity of these mechanisms, whether $O(n^2)$ for BFT, $O(\sqrt{n}\log n)$ for sortition, or $O(\text{difficulty})$ for PoW, directly constrains the feasible committee size, which in turn bounds either throughput (small committee ⇒ low parallelism) or decentralization (large committee ⇒ centralizing hardware/bandwidth requirements).

NeuroGraph exists because there is a logically distinct path to distributed agreement: *emergent consensus through local neural computation.* If each node can independently arrive at a prediction that—by virtue of observing the same reality—is statistically consistent with the predictions of all other honest nodes, then the global consensus value does not require iterative voting or multi-round agreement messages. It can be *computed* as the reputation-weighted median of local predictions, an $O(n \log n)$ operation under the current sorting-based implementation that requires a single broadcast of each node's prediction vector and no iterative agreement protocol.

This architectural shift changes the trade-offs underlying the Trilemma as follows:

1. **Scalability** is achieved through *sharding with independent consensus*: each of the 333 shards runs its own emergent-consensus process in parallel, yielding $O(\text{shard\_count} \cdot n_{\text{shard}} \log n_{\text{shard}})$ aggregate complexity for $n_{\text{shard}} = 8$ nodes per shard. The per-shard consensus cost is $O(8 \log 8) = O(24)$—a small constant—and cross-shard transactions are coordinated via the two-phase Lock-Commit protocol (Section 7.2) with a hard timeout, avoiding the global-lock bottlenecks that limit sharded BFT systems.

2. **Decentralization** is preserved because the per-node computational requirement is minimal: each node maintains a local AdaptiveDag with $O(1)$ prediction computation per step and broadcasts a single 4-dimensional prediction vector (∼32 bytes). No committee election, no multi-round voting, and no economic stake requirement is needed for consensus participation. The FCFS slot admission with progressive PoW (Section 8) prevents Sybil[2]

[1] Micali, Rabin & Vadhan, "Verifiable Random Functions," *FOCS*, 1999 [13].
[2] Douceur, "The Sybil Attack," *IPTPS*, 2002 [16].

flooding while keeping honest-node admission costs bounded at ∼5 hours of mining for the first tier.

3. **Security** is maintained not through the $f < n/3$ BFT bound or economic finality, but through a *statistical separation guarantee*: honest nodes' predictions converge (driven by Hebbian learning with high α), while attackers' predictions diverge (forced to low α), creating a monotonically widening reputation gap that the dual-EMA engine amplifies over time. The cluster-aware weight cap of 30% bounds any coordinated group's influence (Mechanism Invariant 1), and the 20:1 adaptive learning-rate asymmetry (Mechanism Invariant 2) empirically accelerates honest convergence relative to adversarial adaptation. Within the classical $f < n/3$ bound, these mechanism invariants provide defense-in-depth; beyond $n/3$, attacker detection remains empirically effective (Section 14), though formal consensus safety is not guaranteed.

We emphasize that NeuroGraph does not *solve* the Blockchain Trilemma in the mathematical sense—no protocol can eliminate the underlying trade-offs without introducing additional assumptions or constraints. Instead, NeuroGraph *sidesteps* the Trilemma's structural root by replacing explicit agreement with emergent computation, introducing a different set of assumptions and trade-offs:

- The emergent-consensus approach assumes that honest nodes share a *common observational reality* that their Hebbian predictors can learn. In simulation, this is modelled by a synthetic validation signal (Section 2.2); in production, it would be grounded in application-specific observables (transaction patterns, market data, etc.).
- The security model is validated through a 37-attacker simulation taxonomy (Section 12), not a formal BFT safety proof. The distinction between empirical detection effectiveness and formal safety is discussed in Section 15.1.
- The cross-shard protocol inherits the two-phase commit assumption common to all sharded ledgers: if the locking shard becomes partitioned, the locked funds are unavailable until the receipt expires (1,000 steps). This is a standard availability trade-off of two-phase cross-shard coordination: safety of the locked state is preserved, but liveness of the locked funds depends on receipt expiry.

These trade-offs are explicitly declared rather than hidden, and the three-level guarantee classification (Formally bounded / Empirically validated / Projected) in Section 15.1 ensures that readers can assess the strength of each claim without overstatement.

2 Layer 1: Transaction

The Transaction layer defines the fundamental data structure and cryptographic primitives that flow through all subsequent layers. Each transaction is a signed, hash-chained record with Ed25519[1] authentication and SHA-512/256 integrity.

Definition 1: Transaction Structure

A transaction $\text{tx} \in \mathcal{T}$ is a 9-tuple:

$$\text{tx} = (s, r, a, f, n, ts, P, \sigma, h) \tag{1}$$

where s is the sender address, r is the receiver address, $a \in \mathbb{Z}_{\geq 0}$ is the amount in ANGP units (1 ANGP $= 10^8$ internal units), $f = \lfloor a/1000 \rfloor$ is the fee (0.1%), n is the nonce, ts is the timestamp, $P \subset \{0,1\}^{256}$ is the set of parent hashes (DAG edges), σ is the Ed25519 signature, and $h = H(s \| r \| a \| f \| n \| ts \| P)$ is the transaction hash.

The hash function H uses SHA-512/256 with binary encoding and 0xFF field separators to prevent cross-field boundary ambiguity. This produces a 32-byte hash from approximately 80 bytes of input (versus approximately 400 bytes from string-based encoding), eliminating approximately 2μs overhead per transaction.

2.1 Transaction Types

Transactions support four kinds via the TxKind enum:

- **Normal**: Standard value transfer
- **Lock**: Cross-shard debit (Phase 1 of cross-shard protocol)
- **Commit{receipt_hash}**: Cross-shard credit (Phase 2)
- **Refund{receipt_hash}**: Automatic refund on receipt expiry

2.2 Ed25519 Batch Verification

Signature verification supports batch processing via `ed25519-dalek::verify_batch` with chunk size 1000, parallelized with Rayon. For n transactions, this reduces verification from $O(n)$ individual verifications to $O(n/1000)$ batch operations, providing approximately 3–5× speedup on multi-core systems.

The deterministic ordering key for batch proposals is computed as:

$$\text{order}(\text{tx}) = H(h \| n) \tag{2}$$

ensuring that all nodes produce identical transaction ordering within a batch, which is critical for deterministic state transitions and consistent reward distribution.

[1] Bernstein et al., "Ed25519: High-speed high-security signatures," 2012 [14].

3 Layer 2: Prediction

The Prediction layer is the cognitive core of NeuroGraph. Each node maintains an *Adaptive Directed Acyclic Graph* (AdaptiveDag) with Hebbian learning that produces a 4-dimensional prediction vector $\mathbf{p} = (p_a, p_f, p_\phi, p_\sigma) \in \mathbb{R}^4$ serving as the node's neural output in the consensus process. The four dimensions represent: (1) *signal amplitude* p_a — the predicted magnitude of the shared sinusoidal signal; (2) *frequency* p_f — the predicted oscillation rate; (3) *phase* p_ϕ — the predicted phase offset; and (4) *noise variance* p_σ — the predicted Gaussian noise level. This 4D structure captures the essential degrees of freedom of the honest signal $s(t) = A\sin(\omega t + \phi) + \mathcal{N}(0, \sigma^2)$, enabling fine-grained anomaly detection across each component independently.

Definition 2: AdaptiveDag

An AdaptiveDag $\mathcal{G} = (V, E, W)$ consists of:

- $V = \{v_0, v_1, \ldots, v_t\}$: ordered sequence of prediction nodes
- $E \subseteq V \times V$: parent-child edges (chain structure in practice)
- $W : E \to \mathbb{R}_{>0}$: Hebbian weight functions

Each node v_i stores a prediction vector $\mathbf{p}_i \in \mathbb{R}^4$, and each edge $(v_i, v_j) \in E$ has weight w_{ij}.

3.1 Hebbian Learning Rule

The Hebbian weight update follows an *adaptive learning rate* formulation:

Protocol Rule 1: Adaptive Hebbian Update

Given a new prediction $\mathbf{v}$ at node v_i with parent v_j having prediction $\mathbf{p}_j$, the weight update is:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \alpha \cdot (\text{sim}(\mathbf{v}, \mathbf{p}_j) - w_{ij}^{(t)}) \tag{3}$$

where the similarity function is:

$$\text{sim}(\mathbf{v}, \mathbf{p}_j) = \frac{1}{1 + \|\mathbf{v} - \mathbf{p}_j\|_2} \tag{4}$$

and the adaptive learning rate is:

$$\alpha = \alpha_{\min} + (\alpha_{\text{base}} - \alpha_{\min}) \cdot \text{agreement}^p \tag{5}$$

with $\alpha_{\min} = 0.005$, $\alpha_{\text{base}} = 0.10$, $p = 2.0$, and:

$$\text{agreement} = \max\left(0,\ 1 - \frac{\|\mathbf{v} - \mathbf{m}\|_2}{\|\text{norm}\|}\right) \tag{6}$$

where $\mathbf{m}$ is the consensus median from the previous step.

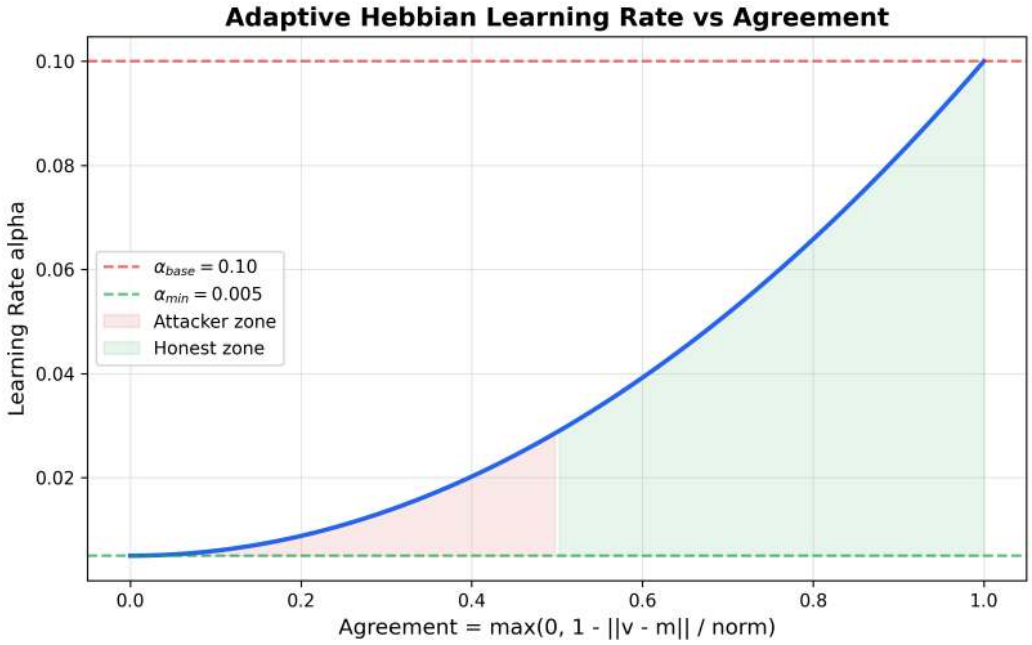

Figure 1: Adaptive Hebbian learning rate α as a function of agreement. Honest nodes (agreement $\to 1$) receive $\alpha \to 0.10$, while attackers (agreement $\to 0$) receive $\alpha \to 0.005$, creating a 20:1 maximum adaptive learning-rate ratio.

This creates a critical feedback loop:

- **High agreement** (honest nodes: $\mathbf{v} \approx \mathbf{m}$): $\alpha \to \alpha_{\text{base}} = 0.10$, enabling rapid consolidation of correct predictions
- **Low agreement** (attackers: $\mathbf{v} \neq \mathbf{m}$): $\alpha \to \alpha_{\min} = 0.005$, preventing consolidation of perturbed predictions

3.2 Prediction Computation

The predicted output of node v_i with parents $\{v_{j_1}, \ldots, v_{j_k}\}$ is the Hebbian-weighted aggregate:

$$\hat{\mathbf{p}}_i = \frac{\sum_{k=1}^{K} w_{ij_k} \cdot \mathbf{p}_{j_k}}{\sum_{k=1}^{K} w_{ij_k}} \tag{7}$$

The honest signal is a 4D sinusoid with Gaussian noise and EMA smoothing:

$$s_d(t) = \beta + A \sin(\omega t + d \cdot \phi_0) + \mathcal{N}(0, \sigma^2), \quad d = 0, 1, 2, 3 \tag{8}$$

with $\beta = 0.5$ (baseline), $A = 0.4$ (amplitude), $\sigma = 0.005$ (noise std), $\phi_0 = 0.1$ (phase offset), and EMA coefficient $\alpha_{\text{EMA}} = 0.3$.

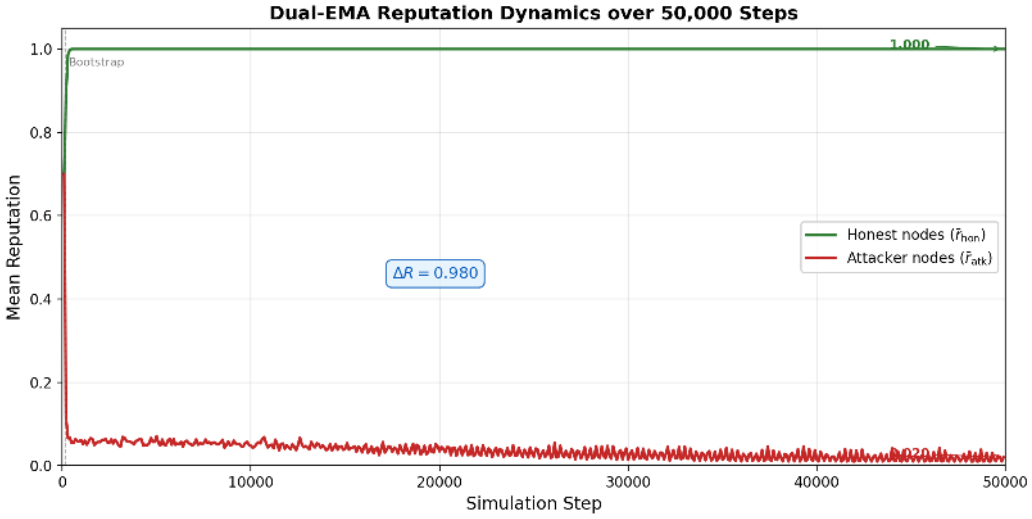

Figure 2: Simulated dual-EMA reputation dynamics over 50,000 steps. Honest nodes (green) rapidly converge to $r \to 1.0$, while coordinated attackers (red) decay to $r \to 0.0$, demonstrating the self-reinforcing feedback loop. The extended simulation confirms that reputation separation is stable and monotonic beyond the initial convergence phase.

4 Layer 3: Consensus

The Consensus layer implements *emergent neural consensus* through reputation-weighted median computation. Unlike BFT protocols that require explicit message-exchange rounds, NeuroGraph's consensus emerges from the statistical aggregation of neural predictions.

Definition 3: Emergent Consensus

Given proposals $\{(\mathbf{p}_1, r_1), \ldots, (\mathbf{p}_N, r_N)\}$ where $\mathbf{p}_i \in \mathbb{R}^4$ is the prediction and $r_i \in [0, 1]$ is the reputation, the consensus median is:

$$\mathbf{m} = \text{WeightedMedian}(\{\mathbf{p}_i\}, \{w_i\}) \tag{9}$$

where w_i are the *cluster-capped* weights derived from raw reputation weights $\hat{w}_i = \max(r_i, 0.01)$.

4.1 Cluster-Aware Weight Capping

To prevent coordinated attacks from dominating the median, a Union-Find-based cluster detection algorithm caps the total weight of any cluster of identical predictions:

$$w_i = \begin{cases} \hat{w}_i \cdot \frac{c_{\text{cap}}}{W_{\text{cluster}}} & \text{if } W_{\text{cluster}} > c_{\text{cap}} \\ \hat{w}_i & \text{otherwise} \end{cases} \tag{10}$$

where $c_{\text{cap}} = W_{\text{total}} \times 0.30$ and W_{cluster} is the total weight of the cluster containing node i. The cluster detection uses an $O(N \log N)$ sort-based algorithm with early exit when all predictions fall within $\varepsilon = 0.0001$ of each other.

4.2 Multi-Component Error

The error assigned to each node combines three independent components:

$$e_i = \underbrace{\tanh\left(\frac{\|\mathbf{p}_i - \mathbf{m}\|_2}{\text{NF}}\right) \cdot s_e}_{\text{neural}} + \underbrace{\frac{\lambda_t}{|T_i|} \sum_{t \in T_i} \mathbb{1}[t \notin T_{\text{common}}]}_{\text{structural}} + \underbrace{\lambda_s \cdot \mathbb{1}[\text{root}_i \neq \text{root}_{\text{common}}]}_{\text{state}} \tag{11}$$

where NF is the normalization factor (median pairwise distance), $s_e = 0.2$ is the soft error scale, $\lambda_t = 0.05$ is the tip difference penalty, $\lambda_s = 0.15$ is the state root penalty, T_i are the proposed tips, and T_{common} are the consensus tips.

4.3 Hybrid Shard Consensus

The hybrid consensus operates at two levels:

1. **Layer 1 — Shard Consensus**: For each shard S_k, compute local consensus $\mathbf{m}_k$ on proposals from nodes in S_k
2. **Layer 2 — Global Anchor**: Compute consensus on shard digests $\{(\mathbf{m}_i, h_i)\}_{i=1}^{K}$ to detect compromised shards

A shard is flagged as *compromised* if $\|\mathbf{m}_k - \mathbf{m}_{\text{global}}\|_2 > 0.3$.

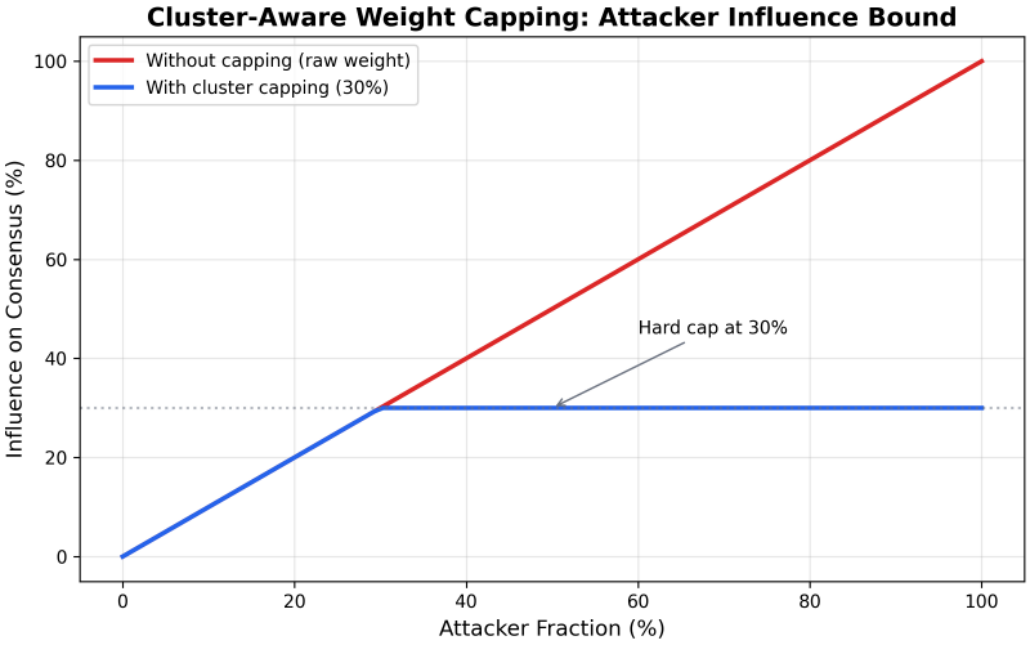

Figure 3: Cluster-aware weight capping bounds attacker influence at 30% regardless of their fraction. Without capping, attackers at 50% would dominate the consensus; with capping, their influence is hard-bounded.

5 Layer 4: Reputation

The Reputation layer implements a dual-Exponential Moving Average (EMA) engine with 13 independent detection mechanisms, consolidated into a single `PeerReputationState` struct for cache locality. Together with the 2 attack detectors in the Security layer (Section 11), the system employs 15 detection mechanisms across Layers 4 and 5. The reputation values are explicitly clamped to $[r_{\min}, 1]$ after each update, where $r_{\min} = 0$ is the theoretical minimum and the practical floor during bootstrap is 0.30.

5.1 Dual-EMA Reputation

Definition 4: Dual-EMA Reputation

The reputation of node i at step t is:

$$r_{\text{fast}}^{(t)} = \alpha_f \cdot c^{(t)} + (1 - \alpha_f) \cdot r_{\text{fast}}^{(t-1)} \tag{12}$$

$$r_{\text{slow}}^{(t)} = \alpha_s \cdot c^{(t)} + (1 - \alpha_s) \cdot r_{\text{slow}}^{(t-1)} \tag{13}$$

$$r^{(t)} = \min(r_{\text{fast}}^{(t)},\ r_{\text{slow}}^{(t)}) \tag{14}$$

where $c^{(t)}$ is the continuous certificate, $\alpha_f = 0.5$ (fast EMA), $\alpha_s = 0.02$ (slow EMA). After computing $r^{(t)}$, the implementation applies an explicit clamp:

$$r^{(t)} \leftarrow \max(0, \min(1, r^{(t)})) \tag{15}$$

This ensures $r^{(t)} \in [0, 1]$ at all times. Without this clamp, a node with $\delta_i = 0$ (perfect prediction) and $\eta > 0$ would exhibit unbounded growth $r(t) \sim r_0(1 + \eta)^t$. The clamp is necessary because the EMA formulation does not intrinsically bound r from above when $c^{(t)} > r^{(t-1)}$.

The continuous certificate function is:

$$c(e) = \begin{cases} 1 - \left(\frac{e}{\theta_+}\right)^{\gamma} & \text{if } e < \theta_+ \\ 0 & \text{if } e \geq \theta_+ \end{cases} \tag{16}$$

where $\theta_+ = 1.2$ is the positive threshold and the exponent $\gamma = 4.27$ provides a smooth decay that distinguishes honest errors ($e \ll \theta_+$) from attacker errors ($e \approx \theta_+$).

5.2 Thirteen Detection Mechanisms

Table 1 summarizes the 13 independent detection mechanisms applied within the reputation engine's `update_reputation()` method, all operating on the consolidated `PeerReputationState` struct.

Table 1: Reputation Detection Mechanisms (Layer 4, from `reputation.rs`)

#	Mechanism	Window	Penalty
1	Spike Detection	1000 certs	Cap: $\frac{0.5}{1+\text{spikes}}$
2	Strike System	—	$r \leftarrow r \cdot e^{-0.3 \cdot \text{strike}}$, decay 0.999
3	Large Error Immediate	$e > 0.6$	$\times 0.98$
4	Changepoint Detection	10/50	$\times 0.95$
5	Large Error Frequency	50	$\times 0.92$ if > 10 in window
6	Consecutive Bad	—	$\times 0.9$ after 5
7	Bad Frequency	50	$\times 0.95$ if freq > 0.4
8	Any Negative Flag	50	$\times 0.7$ if ≥ 15
9	Alternating Pattern	20	$\times 0.95$ if transitions > 12
10	Large Error Intervals	20 timestamps	$\times 0.6$ if CV < 0.5 & mean > 200
11	Violation Mechanism	—	$r = 0$ after 2000 consecutive
12	Fault Detection	2000 steps	$\times 0.995$
13	Reputation Floor + Grace Decay	—	Min 0.30; $r_f \times 0.9$, $r_s \times 0.98$

5.3 Reputation Floor

New nodes and recently recovered nodes are protected by a reputation floor of $r_{\text{floor}} = 0.30$ during the bootstrap phase (150 steps) and grace period (200 steps after last seen), preventing death spirals where new honest nodes are immediately penalized before they can establish a track record. This bootstrap mechanism is critical for breaking the consensus-reputation feedback circularity: during the initial 150 steps, all nodes receive near-equal reputation ($r \approx 0.5 + 0.5 \cdot t/150$), allowing the honest majority to establish an initial consensus before attacker reputation can compound. Once bootstrap completes, the honest majority's higher reputation defines the consensus baseline against which attackers are measured.

During the bootstrap phase, malicious nodes benefit from the same floor protection as honest nodes: their reputation cannot drop below $r_{\text{floor}} = 0.30$, and they retain non-negligible influence in the weighted median. This is by design — the alternative (penalizing nodes immediately) would create false positives among honest nodes that are still converging. However, once the bootstrap period ends, attackers diverge rapidly because their predictions systematically deviate from the honest consensus. The grace period (200 steps after last heartbeat) similarly protects recovered or rejoining nodes: a node that re-enters after a temporary absence receives the floor for 200 steps, but if it is an attacker in disguise, its behavioral deviation is detected within this window and its reputation decays below the floor once grace expires. The combination of bootstrap floor and grace-period floor thus provides a controlled safety window: short enough that attackers cannot exploit it indefinitely, long enough that honest nodes are not falsely eliminated during transient convergence.

6 Layer 5: Security

The Security layer provides three complementary mechanisms: Proof of Work identity certificates, VRF-based proposer selection, and the unified anomaly detection system (2 attack detectors, complementing Layer 4's 13 mechanisms for a total of 15 across both layers).

6.1 Attack Detection Mechanisms

The Security layer contains two specialized attack detectors that operate asynchronously in the Slow Lane, sending penalty signals back to the Fast Lane for reputation adjustment:

1. **CoordinationDetector** — Detects clusters of nodes producing identical or near-identical predictions via $O(N)$ fingerprint hashing. When ≥ 2 nodes produce predictions within $\varepsilon_{\text{coord}} = 0.0001$ Euclidean distance, all nodes in the cluster receive a penalty of 0.5. This catches clone attacks, coordinated bias, Sybil clusters with similar predictions, mimicry attacks, and frozen-drift attackers. The $O(N)$ optimization (v3.14) replaces the previous $O(N^2)$ pairwise comparison by quantizing predictions to 64-bit fingerprints and grouping via HashMap, reducing comparison cost from 50M operations (10K nodes) to ∼10K on average.

2. **AdaptiveDetector** — Detects attackers who misbehave only when the consensus mean is near 0.5 (the zone of maximum influence). For each node, a sliding window of 80 observations (`ADAPTIVE_DETECTION_WINDOW`) tracks the correlation between prediction error and $|\bar{c} - 0.5|$. A node is flagged as adaptive through a two-tier detection rule:

   - **Primary rule:** In-zone mean error $\bar{e}_{\text{in}} > 0.03$ and out-zone mean error $\bar{e}_{\text{out}} < 0.03$ — penalty 0.5.
   - **Ratio rule:** $\bar{e}_{\text{in}} > 0.03$, $\bar{e}_{\text{out}} > 0.001$ (guard against near-zero denominator), and $\bar{e}_{\text{in}}/\bar{e}_{\text{out}} > 2.0$ with ≥ 5 in-zone observations — penalty $0.5 \times 0.8 = 0.4$ (reduced to limit false positives from honest nodes with elevated noise).

   This catches classic adaptive attackers, sleepers at activation, byzantine nodes with intermittent errors, noise injectors, and jump attackers.

Both detectors feed into the unified anomaly score computation:

$$a_i = 0.30 \cdot e_i + 0.20 \cdot d_i + 0.35 \cdot c_i + 0.15 \cdot t_i \tag{17}$$

where e_i is the error signal, d_i the drift signal, c_i the coordination signal, and t_i the temporal inconsistency signal, with $a_i > 0.7$ indicating a likely attacker.

6.2 Proof of Work Identity Certificates

Each node must mine a PoW certificate at creation:

$$\text{PoW}(\text{id}, d): \quad \text{find } n \text{ such that } H(\text{id} \| : \| n) < 2^{256-4d} \tag{18}$$

where difficulty $d = 6$ (testnet), requiring $16^6 \approx 16.7$M hashes (∼100ms). This low difficulty was chosen during protocol development to facilitate rapid testing and iteration. For mainnet deployment, the difficulty will be calibrated to achieve a target per-node mining time of ∼5–12 hours depending on slot tier, making identity creation a substantive computational commitment. The input uses `id:n` order with a delimiter to prevent input malleability (without delimiter, `"node1"||23` = `"node12"||3`). The PoW nonce is included in every DagProposal and verified by receivers, preventing Sybil attacks from creating arbitrary identities without computational work.

6.3 VRF Proposer Selection

Proposers are selected via Verifiable Random Function with reputation weighting:

Definition 5: VRF Proposer Selection

For step t and shard s, node i computes:

$$\text{output}_i = \text{VRF}(\text{sk}_i, \text{“propose\_step\_}t\text{\_shard\_}s\text{”}) \tag{19}$$

$$w_i = f_{\text{floor}} + (1 - f_{\text{floor}}) \cdot \min(r_i, r_{\text{cap}}) \tag{20}$$

$$\text{threshold}_i = \text{VRF\_threshold}(f_{\text{base}} \cdot w_i) \tag{21}$$

Node i is a proposer iff $\text{output}_i < \text{threshold}_i$, with $f_{\text{base}} = 0.01$, $f_{\text{floor}} = 0.20$, $r_{\text{cap}} = 1.5$.

The target number of proposers per step scales as $O(\sqrt{N})$:

$$k_{\text{target}} = \max(3, \lceil \sqrt{N} \rceil) \tag{22}$$

For 2664 nodes: $k_{\text{target}} = \lceil \sqrt{2664} \rceil = 52$ proposers per step.

The VRF output is computed as SHA-512($0x01 \| \text{sk} \| \text{message}$)$[0:32]$ with Ed25519 proof, ensuring unpredictability and verifiable uniqueness.

7 Layer 6: Network

The Network layer provides a trait-based abstraction over two implementations: TCP gossip (default, no external dependencies) and libp2p GossipSub[1] (feature-gated).

7.1 Message Types

Seven message types flow through the network:

1. **DagProposalMessage**: Complete DAG proposal with prediction, tips, state root, and VRF proof
2. **TransactionMessage**: Signed transaction with sender metadata
3. **ConflictResolutionMessage**: Signal for double-spend conflict resolution
4. **SnapshotRequest/Response**: State synchronization for new nodes
5. **ReputationReport**: Legacy reputation broadcast
6. **FinalizedBatchMessage**: Cross-node ledger sync (v3.11.0+)

7.2 Asynchronous Networking

The `AsyncNetwork` module (v3.9+) provides tokio-based asynchronous networking with bounded channels, replacing the legacy thread-per-connection TCP server. The channel architecture uses `flume` for lock-free MPMC communication between the network layer and the main consensus loop.

7.3 Finalized Batch Gossip

When a node finalizes a batch locally (reaching BATCH_APPROVAL_THRESHOLD = 0.7), it broadcasts a `FinalizedBatchMessage` containing the transactions and reward map. Receivers apply these directly to their ledger, bypassing redundant local consensus since the sender has already achieved the approval threshold. This prevents balance desynchronization across nodes.

7.4 Double-Spend Conflict Resolution

When two transactions from the same sender with conflicting nonces (double-spend) are detected, the conflict resolution mechanism collects reputation-weighted signals from observing nodes. Each observer records a resolution signal tagged with its reputation value; duplicate signals from the same observer are rejected. A conflict is resolved when the reputation-weighted fraction of signals favoring one transaction exceeds 60%, or times out after $W_{\text{conflict}} = 20$ steps. Resolved conflicts are pruned to maintain bounded memory. This mechanism operates independently of the Hebbian-inspired consensus and handles the specific case of concurrent double-spend attempts that reach different shards.

[1] Vyzo et al., "GossipSub: An extensible, scalable pubsub protocol," libp2p specification, 2020 [15].

8 Layer 7: Sharding

The Sharding layer partitions the state space into $N_{\text{shards}} = 333$ parallel processing shards, each capable of local consensus, while maintaining global security through the hybrid consensus mechanism. Shards are not fully independent—cross-shard transactions require coordination via the Lock-Commit protocol (Section 7.2).

8.1 Deterministic Shard Assignment

Shard assignment is deterministic via cryptographic hash:

$$\text{shard}(\text{addr}) = H_{512/256}(\text{addr})[0:4] \bmod 333 \tag{23}$$

This ensures: (1) the same address always maps to the same shard, (2) the distribution is uniform (SHA-512/256 is a cryptographic hash), and (3) no coordination is needed for shard lookup.

Adversarial consideration. An attacker who can choose sender addresses could attempt a shard concentration attack by generating addresses that hash to a target shard. However, the preimage resistance of SHA-512/256 makes targeted address generation computationally infeasible: finding an address mapping to a specific shard requires $O(333) = O(2^{8.4})$ attempts on average, which is trivial per address but becomes significant when attempting to concentrate many identities in a single 8-node shard. The FCFS slot admission with progressive PoW (Section 8) provides the primary defense: even if an attacker generates addresses for a target shard, they still need PoW certificates and available slots. With only 8 slots per shard, the attacker can occupy at most 8 positions before being blocked by the standby queue. Future work may include epoch-based reshuffling for additional robustness.

8.2 Cross-Shard Protocol

Cross-shard transactions execute via a two-phase Lock-Commit protocol:

Algorithm 1 Cross-Shard Lock-Commit

Phase 1 (Lock, shard of sender):
  Debit sender by $a + f$
  Generate receipt $\rho = (\text{hash}, s, r, a, \text{shard}_s, \text{shard}_r, \text{step}, n)$
  Broadcast ρ via gossip
Phase 2 (Commit, shard of receiver):
  Verify ρ is valid and not expired
  Credit receiver by a
  Fee f remains with proposer in shard_s
On Expiry (step $>$ created $+ 1000$):
  Refund a to sender (fee is not refunded)

Receipt validity requires: $\text{shard}(s) = \text{from\_shard}$, $\text{shard}(r) = \text{to\_shard}$, $\text{from\_shard} \neq \text{to\_shard}$, $a > 0$.

9 Layer 8: Slot

The Slot layer manages node admission through a First-Come-First-Served (FCFS) mechanism with 2664 active slots (333 shards × 8 nodes/shard), anti-whale PoW difficulty, and automatic standby promotion.

9.1 FCFS Admission Protocol

Definition 6: Slot Admission

A node i is admitted to an active slot if:

1. A free slot exists: $\text{admit}(i) \to (\text{slot\_id}, \text{active})$
2. No free slot: enters standby queue (FIFO, max 2000 entries)
3. Grace period: if i had a slot that timed out within 1 hour, reclaim same slot

9.2 Anti-Whale Progressive PoW

The PoW difficulty increases with the number of occupied slots. The current testnet uses low difficulties for rapid iteration; mainnet deployment will use calibrated difficulties targeting 5–12 hours of mining per node depending on slot tier:

Table 2: Progressive PoW Difficulty (Mainnet Targets)

Slots Occupied	Testnet Diff.	Mainnet Target	Testnet Time	Mainnet Time
0–531 (532 slots, 20%)	6	calibrated	∼5 seconds	∼5 hours
532–1331 (800 slots, 30%)	7	calibrated	∼10 seconds	∼8 hours
1332–2663 (1332 slots, 50%)	8	calibrated	∼20 seconds	∼12 hours

On mainnet with target per-node mining times of 5h/8h/12h across the three tiers, the total sequential mining cost reaches approximately 1,044 days, making Sybil attacks prohibitively expensive. The minimum per-node mining time on mainnet is ∼5 hours (tier 1), ensuring that honest nodes can join within half a day while an attacker seeking to dominate the network faces a cumulative cost of years to fill all slots. The mainnet difficulty will be finalized at launch based on observed network hash rate to achieve the target per-node mining times.

9.3 Heartbeat and Timeout

Active nodes must send heartbeats every 10 minutes. Missing a heartbeat for 60 minutes (1 hour) triggers slot release and standby promotion within 30 seconds, ensuring recovery from node failures while tolerating transient network disruptions.

10 Layer 9: Finality

The Finality layer implements a Batch DAG with epoch-based organization, gap detection for sync, and incremental pruning for bounded memory usage.

10.1 Batch DAG Structure

Finalized batches form a hash-chained DAG where each batch references its parent:

$$B_i = (h_i, e_i, \text{tx\_count}, h_{\text{parent}}) \tag{24}$$

The `verify_batch_chain` function traverses the chain from the latest batch to genesis, verifying that: (1) every parent is finalized, and (2) no parent has more than one child (no forks). This ensures that each shard maintains a linear batch chain, which is critical for deterministic state progression.

10.2 Incremental Pruning

The ledger maintains a maximum of 1,000,000 entries with three-level pruning:

Table 3: Incremental Pruning Levels

Fill Level	Batches Pruned/Call	Entries Removed	Time Budget
< 900K	0	—	—
900K–980K	1	$\sim$2000	600μs
980K–1M	2	$\sim$4000	600μs
> 1M	3	$\sim$6000	600μs

This spreads what would be a $\sim$55ms burst prune over approximately 50 steps as $\sim$1ms each, keeping the ledger hovering between 900K and 1M entries.

10.3 Three-Lane Transaction Processing

Transactions are classified into three lanes based on a composite priority score that combines transaction-level and node-level features:

$$\text{score} = 0.30 \cdot s_a + 0.25 \cdot s_f + 0.15 \cdot s_n + 0.30 \cdot s_r \tag{25}$$

where s_a is the *amount score* (logistic function of the transaction amount — very large or very small amounts score higher, flagging potential dust attacks or whale transfers), s_f is the *fee score* (linear function of the fee relative to the amount — underpaid fees suggest spam or low-priority transactions), s_n is the *nonce score* (parabolic function of the nonce gap — large gaps indicate missing or delayed transactions that may be part of a double-spend attempt), and $s_r = 1 - \text{rep}$ is the *reputation score* (low reputation = high priority for inclusion). The weights (0.30, 0.25, 0.15, 0.30) balance the influence of each factor, with amount and reputation weighted most heavily because they carry the strongest security signal. Lane assignment:

Table 4: Three-Lane Classification

Lane	Score Range	Batch Size	Threshold	Priority
Fast	$[0, 0.25)$	256	0.25	High
Medium	$[0.25, 0.60)$	128	0.60	Medium
Slow	$[0.60, 1.0]$	32	—	Low

Fast lane processes first, then Medium, then Slow. Deterministic ordering within each lane uses FNV-1a hash (∼10ns vs ∼1500ns for SHA-512/256). FNV-1a is retained in production because the inputs being ordered (transaction hashes) are already SHA-512/256 digests, so FNV-1a's collision resistance over already-hashed inputs is sufficient for deterministic ordering. This does not affect security: a collision in FNV-1a over SHA-512/256 outputs would require a SHA-512/256 collision first.

11 Unified Anomaly Detection

The attack detection system in Layer 5 combines two complementary detectors into a unified anomaly score, contributing 2 detectors (Coordination and Adaptive) that complement the 13 mechanisms in Layer 4, totaling 15 detection mechanisms across both layers.

11.1 Coordination Detector

The CoordinationDetector identifies clusters of nodes with identical predictions using fingerprint hashing, achieving $O(N)$ average-case complexity (down from $O(N^2)$ in the pairwise comparison approach):

1. Quantize each 4D prediction to a 64-bit fingerprint
2. Group by fingerprint via HashMap (automatic collision grouping)
3. Verify Euclidean distance only within collision buckets (normally 0 or very few)
4. Flag nodes in clusters ≥ 2 within $\varepsilon = 0.0001$

11.2 Adaptive Detector

The AdaptiveDetector catches attackers that misbehave only near consensus = 0.5:

$$\text{detected} = (\bar{e}_{\text{in}} > \theta_{\text{low}}) \wedge (\bar{e}_{\text{out}} < \theta_{\text{low}}) \vee \left(\frac{\bar{e}_{\text{in}}}{\bar{e}_{\text{out}}} > 2.0 \wedge n_{\text{in}} \geq 5 \right) \tag{26}$$

where $\bar{e}_{\text{in}}$ and $\bar{e}_{\text{out}}$ are mean errors in and out of the adaptive zone ($|c - 0.5| < 0.20$), $\theta_{\text{low}} = 0.03$.

11.3 Unified Anomaly Score

The combined anomaly score is a weighted sum:

$$A = 0.30 \cdot s_{\text{error}} + 0.20 \cdot s_{\text{drift}} + 0.35 \cdot s_{\text{coord}} + 0.15 \cdot s_{\text{temporal}} \tag{27}$$

where s_{coord} has the highest weight (0.35) because coordination is the most dangerous attack pattern. A node with $A > 0.7$ is classified as a likely attacker. Attack detection penalties are applied via the `apply_penalty()` method, which multiplies reputation by $(1 - \text{penalty})$ with a cap of 0.5 per event to ensure recoverability from false positives.

12 Security Analysis: 37-Attacker Model

The system is validated against 37 distinct attacker types organized in three tiers:

Table 5: Complete ANGP attacker taxonomy with detection characteristics

ID	Name	Strategy	Category	Difficulty
T0	Random-Noise	Uniform $[-4, +4]$ perturbation	Disruption	Low
T1	Mimicry-300	300 honest steps, then noise ($\sigma = 0.8$)	Evasion	Medium
T2	Mimicry-500	500 honest steps, then noise ($\sigma = 1.2$)	Evasion	Medium
T3	Adaptive-RepAware	40% attack / 5% recovery ratio	Adaptive	High
T4	Coordinated-Bias	All add same bias +0.25	Coordination	Medium
T5	Gaussian	Gaussian noise $\mathcal{N}(0, 2.0)$	Disruption	Low
T6	FlipFlop	100 honest / 100 attack cycles	Oscillation	Medium
T7	Sleeper	2000 honest steps, then drift	Stealth	High
T8	Progressive-Drift	+0.0002/step degradation	Slow Attack	Very High
T9	Outlier-Burst	95% honest, 5% outlier ± 5	Burst	Medium
T10	Clone-Copy	Copy honest prediction + micro-noise	Impersonation	Medium
T11	Byzantine	Per-dimension random chaos	Byzantine	Low
T12	Sybil-Cluster	5 identities, coordinated bias 0.03	Sybil	High
T13	Rep-Farmer	Farm reputation for 5K steps, then attack	Exploitation	Very High
T14	Oscillating-Drift	Sinusoidal drift pattern	Oscillation	High
T15	Colluding-Committee	20 nodes coordinate together	Coordination	High
T16	Slow-Poison	99.9% valid, 0.1% wrong predictions	Subversion	Very High
T17	Eclipse	Control victim's neighbors	Topology	Extreme
T18	Maj-Ref-Manip	60% same bias +0.01	Manipulation	Very High
T19	Sybil-Replace	Eliminated $\rightarrow$ new identity, reputation reset	Sybil	Extreme
T20	Patient-Byz	5000 perfect honest steps, then full attack	Stealth	Extreme
T21	Threshold-Gamer	Stay below all detection thresholds	Evasion	Extreme
T22	True-Fdbk-Adaptive	Observe own reputation, self-adjust	Adaptive	Extreme
T23	Rep-Gradient	Learn reputation function via probing	Modeling	Extreme
T24	Detector-Mimicry	Mimic honest mean/variance/model	Evasion	Extreme
T25	Distrib-Influence	100 nodes $\times$ small bias, collective push	Coordination	Very High
T26	Anti-Coordination	Same goal, different predictions	Evasion	Very High
T27	Rep-Camouflage	Excellent/attack cycles, aggregate management	Oscillation	Extreme
T28	Long-Poison	0.1% wrong over 5K/6K/7K steps	Subversion	Extreme
T29	Honest-Mal-Switch	Random mode switches, no periodic pattern	Unpredictable	Extreme
T30	Thr-Boundary	Live at threshold $-\varepsilon$, dynamic ε	Evasion	Extreme
T31	Recovery-Exploit	Attack $\rightarrow$ recover $\rightarrow$ attack cycles	Exploitation	Extreme
T32	Sybil-Cycling	Cycle through identities, transfer behavior	Sybil	Extreme
T33	Collud-Honest-Maj	30% attack: 10% aggressive + 20% camouflage	Coordination	Extreme
T34	Consensus-Targeted	Optimize $\lvert C_{\text{attack}} - C_{\text{honest}} \rvert$	Manipulation	Extreme
T35	Multi-Vector (BOSS)	Dynamically select best strategy	Adaptive	Extreme
T36	Worst-Case-Coordinated	Perfect coordination, diverse predictions	Coordination	Extreme

12.1 Key Security Properties

Mechanism Invariant 1: Cluster Influence Bound

For any fraction f of coordinated attackers with identical predictions, the cluster-aware weight capping ensures:

$$\frac{W_{\text{attackers}}}{W_{\text{total}}} \leq \min\left(f,\ 0.30\right) \tag{28}$$

regardless of the number of Sybil identities created. This bounds the attackers' *influence* on the consensus median. We emphasise that this is an **influence bound, not a Byzantine-safety theorem**: it limits the weight attackers can exert on the weighted median but does not, by itself, guarantee consensus correctness under arbitrary Byzantine behaviour. Formal safety within the classical BFT bound ($f < n/3$) remains an open theoretical result.

Mechanism Invariant 2: Adaptive Learning-Rate Asymmetry

The adaptive α creates a maximum learning-rate ratio:

$$\frac{\alpha_{\text{honest}}}{\alpha_{\text{attacker}}} \geq \frac{\alpha_{\text{base}}}{\alpha_{\text{min}}} = \frac{0.10}{0.005} = 20 \tag{29}$$

This bounds the *learning-rate asymmetry* at 20×. Whether honest nodes *converge* 20× faster is an emergent property of the full system dynamics and has been observed empirically in simulation (Section 14), but is not formally proven as a convergence-speed theorem.

Mechanism Invariant 3: Anti-Whale Slot Cost

The total mining time to occupy all 2664 slots on mainnet (target per-node mining times 5h/8h/12h across the three slot tiers) is:

$$T_{\text{total}} = 532 \times 5\text{h} + 800 \times 8\text{h} + 1332 \times 12\text{h} = 25{,}044\text{h} \approx 1{,}044 \text{ days} \tag{30}$$

The per-node mining time ranges from ∼5 hours (tier 1) to ∼12 hours (tier 3), well within the 12-hour upper bound for honest node admission. Honest nodes need mine only one slot to join; only attackers seeking to fill many slots face the cumulative cost. The testnet uses difficulties 6/7/8 for development velocity, yielding approximately 10 hours total.

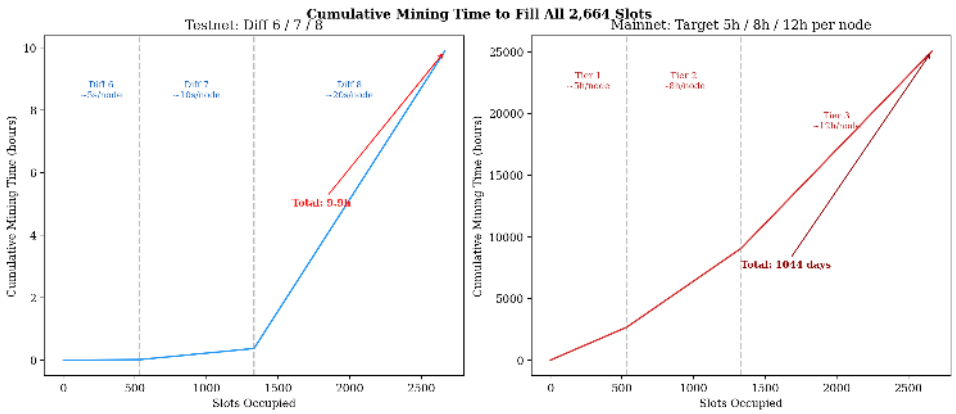

Figure 4: Cumulative mining time to fill all 2664 slots. Left: testnet (Diff 6/7/8, total ∼10h). Right: mainnet (target 5h/8h/12h per node, total ∼1,044 days). The mainnet difficulty will be calibrated at launch based on observed hash rate.

13 Performance Characteristics

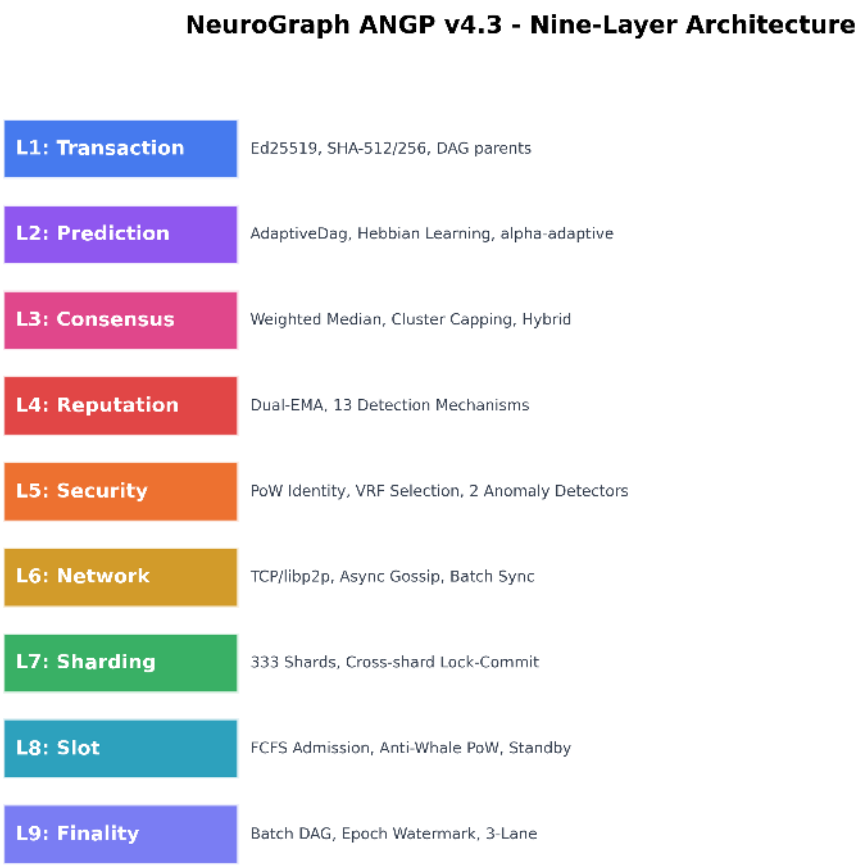

Figure 5: NeuroGraph ANGP v4.3.1 nine-layer architecture. Each layer is independently specified and interacts with adjacent layers through well-defined interfaces. Layer 4 (Reputation): 13 detection mechanisms. Layer 5 (Security): 2 attack detectors (CoordinationDetector, AdaptiveDetector).

Table 6 summarizes the key system parameters extracted from the codebase.

Table 6: System Configuration Parameters (from `config.rs` and `reputation.rs`)

Parameter	Symbol	Value
Prediction dimension	dim	4
Hebbian base learning rate	α_{base}	0.10
Hebbian min learning rate	$\alpha_{\min}$	0.005
Agreement power	p	2.0
Number of shards	N_{shards}	333
Nodes per shard	—	8
Max active nodes	—	2664
Standby queue size	—	2000
Bootstrap steps	—	150
Grace period	—	200
Reputation filter threshold	—	0.9
Cluster weight cap ratio	—	0.30
Coordination epsilon	$\varepsilon_{\text{coord}}$	0.0001
Adaptive zone width	—	0.20
Adaptive detection window	—	80
Fast EMA alpha	α_f	0.5
Slow EMA alpha	α_s	0.02
Positive threshold	θ_+	1.2
Certificate exponent	γ	4.27
Soft error scale	s_e	0.2
State root penalty	λ_s	0.15
Tip diff penalty	λ_t	0.05
VRF base fraction	f_{base}	0.01
VRF reputation floor	f_{floor}	0.20
VRF reputation cap	r_{cap}	1.5
PoW difficulty (testnet)	d	6
PoW difficulty (mainnet)	d	calibrated at launch
Fee divisor	—	1000 (0.1%)
Genesis balance	—	1000 ANGP
Epoch length	—	1000
Finalization interval	—	5
Batch approval threshold	—	0.7
Max tx batch size	—	4096
Mempool max size	—	10,000
Max ledger entries	—	1,000,000
Cross-shard receipt expiry	—	1000 steps
Slot timeout	—	3,600,000 ms (1h)
Heartbeat interval	—	600,000 ms (10min)
Slot reclaim grace	—	3,600,000 ms (1h)

14 Simulation Results

In this section we present comprehensive results from 22 simulation runs spanning three step-count regimes: 10 K steps (9 runs at attacker fractions 10% – 90%), 30 K steps (9 runs at 10% – 90%), and extended runs at 50 K (0%, 50%, and 90% attackers) and 100 K (10% attackers). Each run simulates a network of 2 664 nodes (333 shards × 8 nodes/shard) with up to 37 distinct attacker types.

14.1 Security Validation

Table 7 presents security metrics for 10 K-step runs, Table 8 for 30 K-step runs, and Table 9 for the extended runs (50 K/0%, 50 K/50%, 50 K/90%, and 100 K/10%). Across all configurations, the false-positive rate remains at 0%, and attacker-block rates consistently exceed 99%.

Table 7: Security validation metrics for 10 K-step runs.

Atk%	Honest	Attackers	Eliminated	DS Blocked	Theft Blocked	FPR	ABR	Elim. Rate	TPS
10%	2398	266	260	100%	100%	0.0%	97.7%	97.7%	101,101
20%	2132	532	524	100%	100%	0.0%	98.7%	98.5%	115,139
30%	1865	799	789	100%	100%	0.0%	99.1%	98.7%	115,052
40%	1599	1065	1057	100%	100%	0.0%	99.3%	99.2%	111,201
50%	1332	1332	1326	100%	100%	0.0%	99.5%	99.5%	115,812
60%	1066	1598	1588	100%	100%	0.0%	99.4%	99.4%	117,293
70%	800	1864	1859	100%	100%	0.0%	99.8%	99.7%	129,165
80%	533	2131	2123	100%	100%	0.0%	99.7%	99.6%	127,568
90%	267	2397	2394	100%	100%	0.0%	99.9%	99.9%	136,709

Table 8: Security validation metrics for 30 K-step runs.

Atk%	Honest	Attackers	Eliminated	DS Blocked	Theft Blocked	FPR	ABR	Elim. Rate	TPS
10%	2398	266	265	100%	100%	0.0%	99.6%	99.6%	101,890
20%	2132	532	532	100%	100%	0.0%	100.0%	100.0%	106,794
30%	1865	799	797	100%	100%	0.0%	99.7%	99.7%	118,992
40%	1599	1065	1064	100%	100%	0.0%	99.9%	99.9%	108,873
50%	1332	1332	1331	100%	100%	0.0%	99.9%	99.9%	119,221
60%	1066	1598	1596	100%	100%	0.0%	99.9%	99.9%	119,269
70%	800	1864	1859	100%	100%	0.0%	99.7%	99.7%	119,282
80%	533	2131	2130	100%	100%	0.0%	100.0%	100.0%	133,375
90%	267	2397	2396	100%	100%	0.0%	100.0%	100.0%	119,776

Table 9: Security validation metrics for extended runs (50 K and 100 K steps).

Steps	Atk%	Honest	Attackers	Eliminated	DS Blocked	Theft Blocked	FPR	ABR	Elim. Rate	TPS
50,000	50%	1332	1332	1332	100%	100%	0.0%	100.0%	100.0%	106,668
50,000	90%	267	2397	2397	100%	100%	0.0%	100.0%	100.0%	105,164
100,000	10%	2398	266	266	100%	100%	0.0%	100.0%	100.0%	96,157

14.2 Reputation Dynamics

Tables 10–12 summarise the reputation statistics of honest and attacker nodes. The separability gap $\Delta R = \bar{r}_{\text{hon}} - \bar{r}_{\text{atk,all}}$ confirms clear discrimination in all adversarial settings.

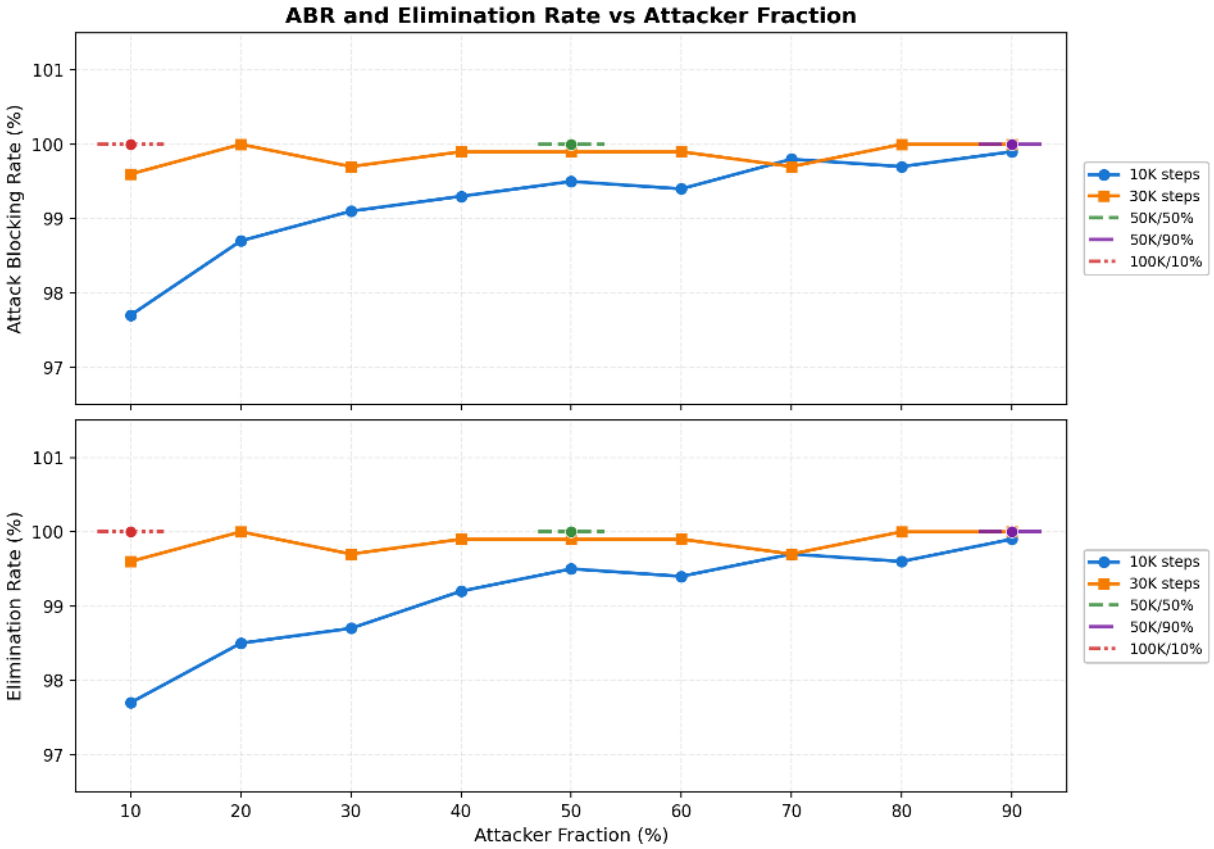

Figure 6: ABR and elimination rate versus attacker fraction across all step regimes.

Table 10: Reputation dynamics for 10 K-step runs.

Atk%	$\bar{r}_{\text{hon}}$	$r_{\text{hon,min}}$	$r_{\text{hon,max}}$	$\bar{r}_{\text{atk,all}}$	Active	Elim	ΔR
10%	0.9989	0.9265	0.9996	0.0310	6	260	0.9679
20%	0.9989	0.9502	0.9996	0.0209	8	524	0.9780
30%	0.9988	0.9270	0.9996	0.0160	10	789	0.9828
40%	0.9989	0.9298	0.9996	0.0125	8	1057	0.9863
50%	0.9988	0.9068	0.9996	0.0099	6	1326	0.9889
60%	0.9988	0.9735	0.9996	0.0117	10	1588	0.9872
70%	0.9988	0.9682	0.9995	0.0079	5	1859	0.9908
80%	0.9987	0.9560	0.9996	0.0086	8	2123	0.9901
90%	0.9981	0.9733	0.9996	0.0064	3	2394	0.9917

Table 11: Reputation dynamics for 30 K-step runs.

Atk%	$\bar{r}_{\text{hon}}$	$r_{\text{hon,min}}$	$r_{\text{hon,max}}$	$\bar{r}_{\text{atk,all}}$	Active	Elim	ΔR
10%	0.9992	0.9540	0.9999	0.0124	1	265	0.9868
20%	0.9992	0.9513	0.9999	0.0072	0	532	0.9920
30%	0.9992	0.9486	0.9999	0.0085	2	797	0.9907
40%	0.9991	0.9016	0.9999	0.0067	1	1064	0.9924
50%	0.9992	0.9489	0.9999	0.0064	1	1331	0.9928
60%	0.9993	0.9595	0.9999	0.0067	2	1596	0.9926
70%	0.9993	0.9790	0.9999	0.0078	5	1859	0.9915
80%	0.9993	0.9803	0.9999	0.0056	1	2130	0.9937
90%	0.9988	0.9608	0.9999	0.0054	1	2396	0.9934

Table 12: Reputation dynamics for extended runs (50 K and 100 K steps).

Steps	Atk%	$\bar{r}_{\text{hon}}$	$r_{\text{hon,min}}$	$r_{\text{hon,max}}$	$\bar{r}_{\text{atk,all}}$	Active	Elim	ΔR
50,000	50%	0.9969	0.9001	0.9999	0.0054	0	1332	0.9915
50,000	90%	0.9563	0.9001	0.9998	0.0044	0	2397	0.9519
100,000	10%	0.9561	0.9402	0.9999	0.0091	0	266	0.9470

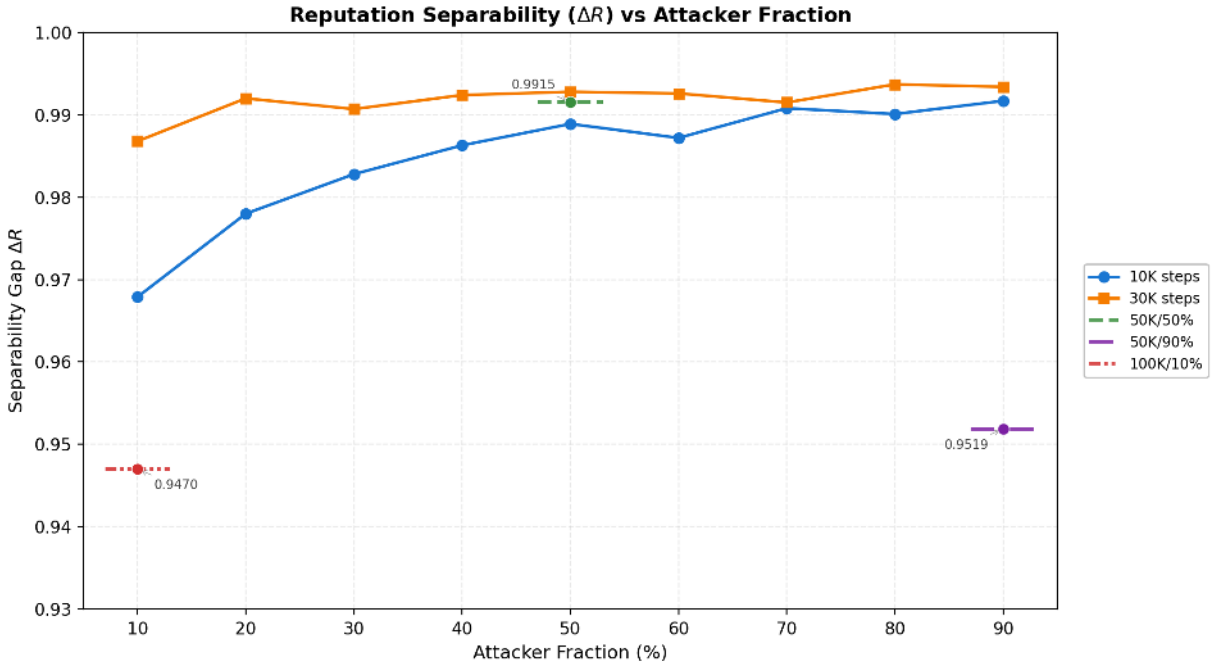

Figure 7: Reputation separability (active and all-node) versus attacker fraction.

14.3 Attack Type Detection

Tables 13 and 14 present per-type elimination rates for **all** tested attacker fractions across the 10 K-step and 30 K-step regimes respectively. Table 15 reports the same for the extended runs (50 K/50%, 50 K/90%, and 100 K/10%). Both standard regimes were tested at nine fractions: 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, and 90%. All extended runs achieve 100% elimination for every attacker type, confirming that with sufficient simulation steps the reputation engine converges completely regardless of attacker fraction or strategy sophistication. Types T0–T15 constitute the primary tier, T16–T21 the intermediate tier, and T22–T36 the advanced tier. In the standard regimes, the vast majority of types achieve 100% elimination across all fractions; exceptions occur primarily in shorter runs where limited simulation steps constrain the reputation convergence window for slow-acting or adaptive strategies.

Extended 50 K/50% run. In the 50 K-step run with 50% attackers, all 37 attacker types (T0–T36) achieved 100% elimination. The false-positive rate was 0.0000% with ABR of 100.0%. The CoordinationDetector identified 707 clusters and the AdaptiveDetector flagged 10 nodes, confirming that even at 50% attacker fraction with sufficient steps, the reputation engine converges completely.

Extended 50 K/90% run. In the 50 K-step run with 90% attackers, all 37 attacker types (T0–T36) achieved 100% elimination. The false-positive rate was 0.0000% with ABR of 100.0%. Honest nodes maintained a mean reputation of 0.9563. The CoordinationDetector identified 1345 clusters and the AdaptiveDetector flagged 251 nodes. Despite the extreme attacker dominance (90%), the system achieves complete attacker elimination with zero false positives, demonstrating that the reputation engine reliably converges even when the honest minority is only 10% of the network.

Extended 100 K/10% run. In the 100 K-step run with 10% attackers, all 37 attacker types (T0–T36) achieved 100% elimination, including the previously resistant Det-Mimic (T24) and Honest-Mal-Switch (T29). This confirms that with sufficient simulation steps, the reputation engine converges even for the most evasive strategies. The detection latency data for this run is reported in the extended latency discussion below.

14.4 Detection Latency and Damage

Tables 16 and 17 detail mean per-type detection and elimination steps, the gap between them, and the wrong-per-node damage averaged across all attacker fractions. Table 18 reports metrics for the 50 K/50% and 50 K/90% extended runs, and Table 19 reports the same metrics for the 100 K/10% extended run. Patient-Byz (T20) consistently exhibits the highest detection step ($\sim$5000) because it waits for the full bootstrap period before acting, while Det-Mimic (T24) shows the highest wrong-per-node damage in the 100K/10% run (129.6) due to its active threshold evasion strategy and the long elimination gap of 10,203 steps.

14.5 Throughput (TPS)

Tables 20–22 report transaction throughput and related metrics for each regime. The extended-run table (Table 22) includes the 50 K/0%, 50 K/50%, 50 K/90%, and 100 K/10% runs with explicit step counts. The 50 K/90% run achieves a TPS of 105,164 with 100% attacker elimination and zero false positives, demonstrating that even under extreme attacker dominance the system maintains substantial throughput.

Table 13: Per-type elimination rate for 10 K-step runs across all attacker fractions.

ID	Name	Cat	10%	20%	30%	40%	50%	60%	70%	80%	90%
T0	`Random Noise`	Disrupt	100%	100%	100%	100%	100%	100%	100%	100%	100%
T1	`Mimicry-300`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T2	`Mimicry-500`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T3	`Adapt-RepAware`	Adaptive	100%	100%	100%	100%	94% (17/18)	100%	100%	100%	100%
T4	`Coord-Bias`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T5	`Gaussian`	Disrupt	100%	100%	100%	100%	100%	100%	100%	100%	100%
T6	`FlipFlop`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T7	`Sleeper`	Stealth	100%	100%	100%	100%	100%	100%	100%	100%	100%
T8	`Prog-Drift`	Slow	100%	100%	100%	100%	100%	100%	100%	100%	100%
T9	`Outlier-Burst`	Burst	100%	100%	100%	100%	100%	100%	100%	100%	100%
T10	`Clone-Copy`	Impers	100%	100%	100%	100%	100%	100%	100%	100%	100%
T11	`Byzantine`	Byz	100%	100%	100%	100%	100%	100%	100%	100%	100%
T12	`Sybil-Cluster`	Sybil	100%	100%	100%	100%	100%	100%	100%	100%	100%
T13	`Rep-Farmer`	Exploit	100%	100%	100%	100%	100%	100%	100%	100%	100%
T14	`Oscill-Drift`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T15	`Collud-Comm`	Coord	100%	93% (13/14)	86% (19/22)	97% (28/29)	98% (53/54)	100%	100%	100%	98% (53/54)
T16	`Slow-Poison`	Subvert	100%	100%	100%	100%	100%	100%	100%	100%	100%
T17	`Eclipse`	Topo	100%	100%	100%	100%	100%	100%	100%	100%	100%
T18	`Maj-Ref-Manip`	Manip	100%	100%	100%	100%	100%	100%	100%	100%	100%
T19	`Sybil-Replace`	Sybil	100%	100%	100%	97% (28/29)	100%	98% (42/43)	100%	100%	100%
T20	`Patient-Byz`	Stealth	100%	100%	100%	100%	100%	100%	100%	100%	100%
T21	`Thr-Gamer`	Evasion	100%	100%	100%	100%	96% (52/54)	100%	98% (49/50)	98% (57/58)	100%
T22	`True-Fdbk`	Adaptive	100%	100%	100%	100%	100%	100%	100%	100%	100%
T23	`Rep-Gradient`	Model	100%	100%	100%	100%	100%	100%	100%	100%	100%
T24	`Det-Mimic`	Evasion	71% (5/7)	64% (9/14)	90% (19/21)	97% (28/29)	94% (34/36)	95% (41/43)	100%	96% (55/57)	100%
T25	`Distrib-Inf`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T26	`Anti-Coord`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T27	`Rep-Camou`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T28	`Long-Poison`	Subvert	100%	100%	100%	100%	100%	100%	100%	100%	100%
T29	`Honest-Mal`	Unpred	43% (3/7)	100%	86% (18/21)	96% (27/28)	94% (34/36)	91% (39/43)	94% (47/50)	96% (55/57)	97% (62/64)
T30	`Thr-Boundary`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T31	`Recov-Expl`	Exploit	100%	100%	100%	100%	100%	100%	100%	100%	100%
T32	`Sybil-Cycle`	Sybil	100%	100%	100%	100%	100%	100%	100%	100%	100%
T33	`Collud-HM`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T34	`Cons-Target`	Manip	100%	100%	100%	100%	100%	100%	100%	100%	100%
T35	`Multi-Vector`	Adaptive	100%	86% (12/14)	90% (19/21)	86% (24/28)	94% (34/36)	93% (40/43)	98% (49/50)	95% (54/57)	98% (63/64)
T36	`WC-Coord`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%

Table 14: Per-type elimination rate for 30 K-step runs across all attacker fractions.

ID	Name	Cat	10%	20%	30%	40%	50%	60%	70%	80%	90%
T0	`Random Noise`	Disrupt	100%	100%	100%	100%	100%	100%	100%	100%	100%
T1	`Mimicry-300`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T2	`Mimicry-500`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T3	`Adapt-RepAware`	Adaptive	100%	100%	100%	100%	100%	100%	100%	100%	100%
T4	`Coord-Bias`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T5	`Gaussian`	Disrupt	100%	100%	100%	100%	100%	100%	100%	100%	100%
T6	`FlipFlop`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T7	`Sleeper`	Stealth	100%	100%	100%	100%	100%	100%	100%	100%	100%
T8	`Prog-Drift`	Slow	100%	100%	100%	100%	100%	100%	100%	100%	100%
T9	`Outlier-Burst`	Burst	100%	100%	100%	100%	100%	100%	100%	100%	100%
T10	`Clone-Copy`	Impers	100%	100%	100%	100%	100%	100%	100%	100%	100%
T11	`Byzantine`	Byz	100%	100%	100%	100%	100%	100%	100%	100%	100%
T12	`Sybil-Cluster`	Sybil	100%	100%	100%	100%	100%	100%	100%	100%	100%
T13	`Rep-Farmer`	Exploit	100%	100%	100%	100%	100%	100%	100%	100%	100%
T14	`Oscill-Drift`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T15	`Collud-Comm`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T16	`Slow-Poison`	Subvert	100%	100%	100%	100%	100%	100%	100%	100%	100%
T17	`Eclipse`	Topo	100%	100%	100%	100%	100%	100%	100%	100%	100%
T18	`Maj-Ref-Manip`	Manip	100%	100%	100%	100%	100%	100%	100%	100%	100%
T19	`Sybil-Replace`	Sybil	100%	100%	100%	100%	100%	100%	100%	100%	100%
T20	`Patient-Byz`	Stealth	100%	100%	100%	100%	100%	100%	100%	100%	100%
T21	`Thr-Gamer`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T22	`True-Fdbk`	Adaptive	100%	100%	100%	100%	100%	100%	100%	100%	100%
T23	`Rep-Gradient`	Model	100%	100%	100%	100%	100%	100%	100%	100%	100%
T24	`Det-Mimic`	Evasion	100%	100%	100%	97% (28/29)	97% (35/36)	98% (42/43)	96% (48/50)	100%	100%
T25	`Distrib-Inf`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T26	`Anti-Coord`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T27	`Rep-Camou`	Oscill	100%	100%	100%	100%	100%	100%	100%	100%	100%
T28	`Long-Poison`	Subvert	100%	100%	100%	100%	100%	100%	100%	100%	100%
T29	`Honest-Mal`	Unpred	100%	100%	95% (20/21)	100%	100%	100%	98% (49/50)	100%	100%
T30	`Thr-Boundary`	Evasion	100%	100%	100%	100%	100%	100%	100%	100%	100%
T31	`Recov-Expl`	Exploit	100%	100%	100%	100%	100%	100%	100%	100%	100%
T32	`Sybil-Cycle`	Sybil	100%	100%	100%	100%	100%	100%	100%	100%	100%
T33	`Collud-HM`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%
T34	`Cons-Target`	Manip	100%	100%	100%	100%	100%	100%	100%	100%	100%
T35	`Multi-Vector`	Adaptive	100%	100%	95% (20/21)	100%	100%	100%	100%	100%	98% (63/64)
T36	`WC-Coord`	Coord	100%	100%	100%	100%	100%	100%	100%	100%	100%

Table 15: Per-type elimination rate for extended runs (50 K and 100 K steps).

ID	Name	Cat	50K/50%	50K/90%	100K/10%
T0	`Random Noise`	Disrupt	100%	100%	100%
T1	`Mimicry-300`	Evasion	100%	100%	100%
T2	`Mimicry-500`	Evasion	100%	100%	100%
T3	`Adapt-RepAware`	Adaptive	100%	100%	100%
T4	`Coord-Bias`	Coord	100%	100%	100%
T5	`Gaussian`	Disrupt	100%	100%	100%
T6	`FlipFlop`	Oscill	100%	100%	100%
T7	`Sleeper`	Stealth	100%	100%	100%
T8	`Prog-Drift`	Slow	100%	100%	100%
T9	`Outlier-Burst`	Burst	100%	100%	100%
T10	`Clone-Copy`	Impers	100%	100%	100%
T11	`Byzantine`	Byz	100%	100%	100%
T12	`Sybil-Cluster`	Sybil	100%	100%	100%
T13	`Rep-Farmer`	Exploit	100%	100%	100%
T14	`Oscill-Drift`	Oscill	100%	100%	100%
T15	`Collud-Comm`	Coord	100%	100%	100%
T16	`Slow-Poison`	Subvert	100%	100%	100%
T17	`Eclipse`	Topo	100%	100%	100%
T18	`Maj-Ref-Manip`	Manip	100%	100%	100%
T19	`Sybil-Replace`	Sybil	100%	100%	100%
T20	`Patient-Byz`	Stealth	100%	100%	100%
T21	`Thr-Gamer`	Evasion	100%	100%	100%
T22	`True-Fdbk`	Adaptive	100%	100%	100%
T23	`Rep-Gradient`	Model	100%	100%	100%
T24	`Det-Mimic`	Evasion	100%	100%	100%
T25	`Distrib-Inf`	Coord	100%	100%	100%
T26	`Anti-Coord`	Evasion	100%	100%	100%
T27	`Rep-Camou`	Oscill	100%	100%	100%
T28	`Long-Poison`	Subvert	100%	100%	100%
T29	`Honest-Mal`	Unpred	100%	100%	100%
T30	`Thr-Boundary`	Evasion	100%	100%	100%
T31	`Recov-Expl`	Exploit	100%	100%	100%
T32	`Sybil-Cycle`	Sybil	100%	100%	100%
T33	`Collud-HM`	Coord	100%	100%	100%
T34	`Cons-Target`	Manip	100%	100%	100%
T35	`Multi-Vector`	Adaptive	100%	100%	100%
T36	`WC-Coord`	Coord	100%	100%	100%

Table 16: Detection latency and damage for 10 K-step runs (mean across all attacker fractions).

ID	Name	Mean Det. Step	Mean Elim. Step	Mean Gap	Mean Wrong/Node
T0	`Random Noise`	271	271	0	3.9
T1	`Mimicry-300`	262	265	3	3.0
T2	`Mimicry-500`	287	292	5	3.1
T3	`Adapt-RepAware`	420	550	131	4.8
T4	`Coord-Bias`	557	553	22	5.7
T5	`Gaussian`	317	317	0	4.1
T6	`FlipFlop`	599	541	0	4.5
T7	`Sleeper`	325	334	10	3.4
T8	`Prog-Drift`	626	684	58	6.7
T9	`Outlier-Burst`	292	301	9	3.1
T10	`Clone-Copy`	321	334	13	3.3
T11	`Byzantine`	256	256	0	3.5
T12	`Sybil-Cluster`	749	1115	366	10.5
T13	`Rep-Farmer`	269	278	10	2.8
T14	`Oscill-Drift`	956	949	52	8.4
T15	`Collud-Comm`	703	793	90	8.8
T16	`Slow-Poison`	310	320	10	3.2
T17	`Eclipse`	954	1162	208	11.1
T18	`Maj-Ref-Manip`	421	444	23	4.3
T19	`Sybil-Replace`	1411	393	0	13.8
T20	`Patient-Byz`	5107	5107	0	52.0
T21	`Thr-Gamer`	1165	1509	344	14.5
T22	`True-Fdbk`	602	870	268	8.3
T23	`Rep-Gradient`	437	468	31	4.3
T24	`Det-Mimic`	1456	1521	66	23.4
T25	`Distrib-Inf`	788	904	116	8.5
T26	`Anti-Coord`	878	1301	423	11.9
T27	`Rep-Camou`	280	290	10	3.0
T28	`Long-Poison`	317	327	10	3.4
T29	`Honest-Mal`	1326	1389	63	22.1
T30	`Thr-Boundary`	435	450	14	6.2
T31	`Recov-Expl`	585	675	90	6.2
T32	`Sybil-Cycle`	347	367	19	3.4
T33	`Collud-HM`	527	551	24	5.7
T34	`Cons-Target`	555	788	233	7.1
T35	`Multi-Vector`	1248	1311	63	18.7
T36	`WC-Coord`	997	1278	281	10.8

Table 17: Detection latency and damage for 30 K-step runs (mean across all attacker fractions).

ID	Name	Mean Det. Step	Mean Elim. Step	Mean Gap	Mean Wrong/Node
T0	`Random Noise`	271	271	0	3.9
T1	`Mimicry-300`	255	257	3	2.9
T2	`Mimicry-500`	275	280	5	3.1
T3	`Adapt-RepAware`	371	569	198	5.0
T4	`Coord-Bias`	520	566	53	5.3
T5	`Gaussian`	299	299	0	4.3
T6	`FlipFlop`	585	535	2	4.8
T7	`Sleeper`	298	309	10	3.1
T8	`Prog-Drift`	732	830	98	7.6
T9	`Outlier-Burst`	275	282	7	3.3
T10	`Clone-Copy`	334	410	76	3.7
T11	`Byzantine`	259	259	0	3.4
T12	`Sybil-Cluster`	750	987	238	9.4
T13	`Rep-Farmer`	315	327	12	3.0
T14	`Oscill-Drift`	786	759	11	7.4
T15	`Collud-Comm`	1110	1089	67	10.4
T16	`Slow-Poison`	289	296	7	2.8
T17	`Eclipse`	1105	1502	396	14.2
T18	`Maj-Ref-Manip`	418	442	24	4.4
T19	`Sybil-Replace`	1183	400	0	16.0
T20	`Patient-Byz`	5109	5109	1	51.5
T21	`Thr-Gamer`	1203	1568	366	14.0
T22	`True-Fdbk`	649	876	227	8.5
T23	`Rep-Gradient`	422	442	20	4.4
T24	`Det-Mimic`	2029	2666	637	30.7
T25	`Distrib-Inf`	691	777	85	7.4
T26	`Anti-Coord`	911	1282	371	11.1
T27	`Rep-Camou`	292	300	8	2.9
T28	`Long-Poison`	307	318	11	3.2
T29	`Honest-Mal`	1898	2264	366	23.8
T30	`Thr-Boundary`	436	442	6	5.9
T31	`Recov-Expl`	603	657	65	6.4
T32	`Sybil-Cycle`	326	403	77	4.0
T33	`Collud-HM`	446	581	135	6.1
T34	`Cons-Target`	711	915	205	7.4
T35	`Multi-Vector`	2032	2175	142	30.0
T36	`WC-Coord`	1074	1241	196	10.9

Table 18: Detection latency and damage for 50 K-step extended runs.

ID	Name	50K/50%				50K/90%			
		Det.	Elim.	Gap	W/N	Det.	Elim.	Gap	W/N
T0	`Random Noise`	271	271	0	3.9	274	274	0	3.9
T1	`Mimicry-300`	254	257	2	2.9	257	259	2	2.8
T2	`Mimicry-500`	274	280	5	3.1	251	254	3	2.9
T3	`Adapt-RepAware`	371	568	197	5.0	349	359	10	3.7
T4	`Coord-Bias`	520	566	53	5.3	483	500	17	4.7
T5	`Gaussian`	298	299	0	4.3	287	288	1	4.1
T6	`FlipFlop`	584	534	2	4.8	429	419	0	3.6
T7	`Sleeper`	298	308	10	3.1	240	242	2	2.6
T8	`Prog-Drift`	732	830	98	7.6	570	613	43	5.6
T9	`Outlier-Burst`	274	282	7	3.3	272	277	5	2.6
T10	`Clone-Copy`	334	410	76	3.7	267	271	4	2.7
T11	`Byzantine`	259	259	0	3.4	249	249	0	3.9
T12	`Sybil-Cluster`	749	987	237	9.4	600	665	65	5.8
T13	`Rep-Farmer`	314	326	11	3.0	260	264	4	3.0
T14	`Oscill-Drift`	786	758	10	7.4	584	630	46	5.7
T15	`Collud-Comm`	1110	1089	67	10.4	864	918	54	9.2
T16	`Slow-Poison`	289	295	6	2.8	279	283	4	2.6
T17	`Eclipse`	1105	1501	396	14.2	887	949	62	9.2
T18	`Maj-Ref-Manip`	417	441	23	4.4	376	399	23	4.0
T19	`Sybil-Replace`	1183	399	0	16.0	1551	333	0	7.9
T20	`Patient-Byz`	5108	5109	0	51.5	5041	5046	5	52.7
T21	`Thr-Gamer`	1202	1568	365	14.0	925	971	46	9.2
T22	`True-Fdbk`	649	876	227	8.5	522	588	66	5.6
T23	`Rep-Gradient`	421	442	20	4.4	305	325	20	3.0
T24	`Det-Mimic`	2028	2666	637	30.7	1265	1322	57	11.5
T25	`Distrib-Inf`	691	776	85	7.4	645	702	57	7.0
T26	`Anti-Coord`	910	1281	371	11.1	906	1134	228	10.6
T27	`Rep-Camou`	292	300	8	2.9	260	263	3	2.5
T28	`Long-Poison`	306	317	11	3.2	244	250	6	2.7
T29	`Honest-Mal`	1897	2263	365	23.8	953	1016	63	9.2
T30	`Thr-Boundary`	436	442	5	5.9	431	445	14	4.8
T31	`Recov-Expl`	602	657	64	6.4	387	438	51	4.4
T32	`Sybil-Cycle`	326	402	76	4.0	294	305	11	3.0
T33	`Collud-HM`	445	581	135	6.1	355	371	16	4.2
T34	`Cons-Target`	710	915	204	7.4	442	484	42	4.7
T35	`Multi-Vector`	2032	2174	142	30.0	1330	2546	1216	25.1
T36	`WC-Coord`	1073	1241	196	10.9	883	991	108	9.2

Table 19: Detection latency and damage for the 100 K/10% extended run.

ID	Name	Det. Step	Elim. Step	Gap	Wrong/Node
	100K / 10% attacker fraction				
T0	`Random Noise`	367	367	0	4.0
T1	`Mimicry-300`	304	312	8	3.6
T2	`Mimicry-500`	388	395	7	4.8
T3	`Adapt-RepAware`	433	451	18	4.2
T4	`Coord-Bias`	1013	1019	6	9.5
T5	`Gaussian`	321	321	0	5.8
T6	`FlipFlop`	1107	939	0	6.9
T7	`Sleeper`	293	312	19	3.0
T8	`Prog-Drift`	905	961	56	7.7
T9	`Outlier-Burst`	322	328	6	2.4
T10	`Clone-Copy`	545	559	14	5.0
T11	`Byzantine`	242	242	0	3.1
T12	`Sybil-Cluster`	792	1514	722	12.4
T13	`Rep-Farmer`	520	541	21	5.1
T14	`Oscill-Drift`	1613	1255	0	10.4
T15	`Collud-Comm`	547	547	0	11.4
T16	`Slow-Poison`	271	291	20	3.4
T17	`Eclipse`	862	927	65	8.9
T18	`Maj-Ref-Manip`	633	672	39	6.7
T19	`Sybil-Replace`	953	288	0	22.7
T20	`Patient-Byz`	5104	5104	0	48.0
T21	`Thr-Gamer`	964	1012	48	8.6
T22	`True-Fdbk`	779	2101	1322	19.6
T23	`Rep-Gradient`	1254	1275	21	12.6
T24	`Det-Mimic`	3515	13718	10203	129.6
T25	`Distrib-Inf`	763	824	61	9.7
T26	`Anti-Coord`	879	1660	781	17.3
T27	`Rep-Camou`	638	667	29	7.1
T28	`Long-Poison`	533	563	30	4.7
T29	`Honest-Mal`	1276	3669	2393	34.6
T30	`Thr-Boundary`	375	375	0	5.6
T31	`Recov-Expl`	630	630	0	7.0
T32	`Sybil-Cycle`	678	722	44	6.1
T33	`Collud-HM`	519	552	33	6.3
T34	`Cons-Target`	1253	1385	132	12.7
T35	`Multi-Vector`	1730	1794	64	17.6
T36	`WC-Coord`	934	1442	508	11.4

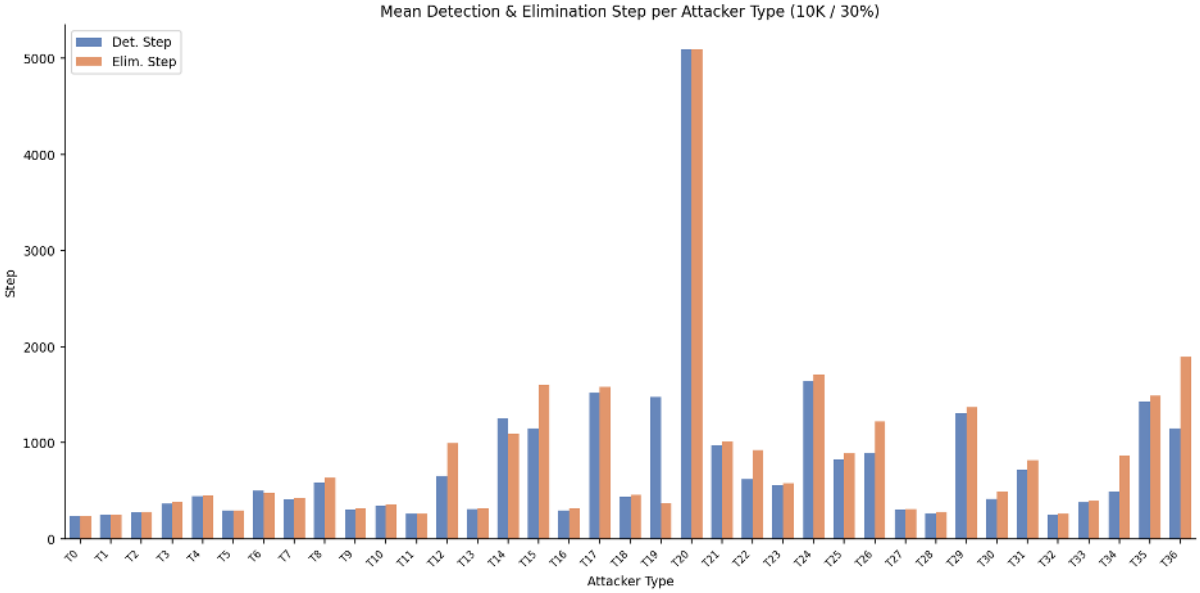

Figure 8: Mean detection and elimination step per attacker type across runs.

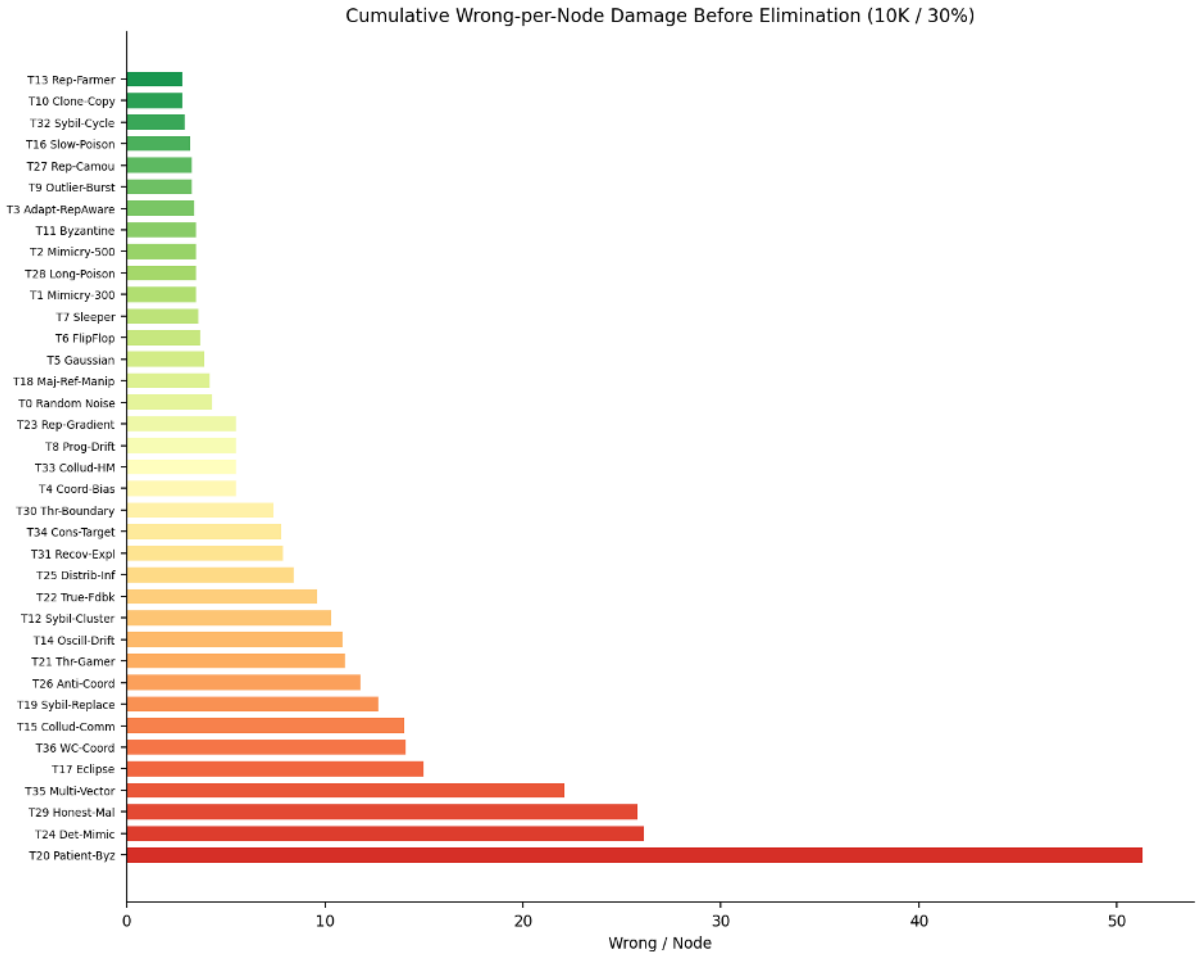

Figure 9: Cumulative wrong-per-node damage before elimination.

Table 20: Throughput metrics for 10 K-step runs.

Atk%	Nodes	Time (s)	Generated	Finalized	Rejected	TPS
10%	2664	197.9	20000000	20010000	0	101101
20%	2664	173.8	20000000	20010000	0	115139
30%	2664	173.8	20000000	20010000	0	115052
40%	2664	179.9	20000000	20010000	0	111201
50%	2664	172.8	20000000	20010000	0	115812
60%	2664	170.6	20000000	20010000	0	117293
70%	2664	154.9	20000000	20010000	0	129165
80%	2664	156.9	20000000	20010000	0	127568
90%	2664	146.4	20000000	20010000	0	136709

Table 21: Throughput metrics for 30 K-step runs.

Atk%	Nodes	Time (s)	Generated	Finalized	Rejected	TPS
10%	2664	588.9	60000000	60030000	0	101890
20%	2664	561.8	60000000	60030000	0	106794
30%	2664	504.2	60000000	60030000	0	118992
40%	2664	551.1	60000000	60030000	0	108873
50%	2664	503.3	60000000	60030000	0	119221
60%	2664	503.1	60000000	60030000	0	119269
70%	2664	503.0	60000000	60030000	0	119282
80%	2664	449.9	60000000	60030000	0	133375
90%	2664	500.9	60000000	60030000	0	119776

Table 22: Throughput metrics for extended runs (50 K and 100 K steps).

Steps	Atk%	Nodes	Time (s)	Generated	Finalized	Rej.	TPS
50,000	50%	2,664	938	100,000,000	100,050,000	0	106,668
50,000	90%	2,664	951	100,000,000	100,050,000	0	105,164
100,000	10%	2,664	2,081	200,000,000	200,100,000	0	96,157

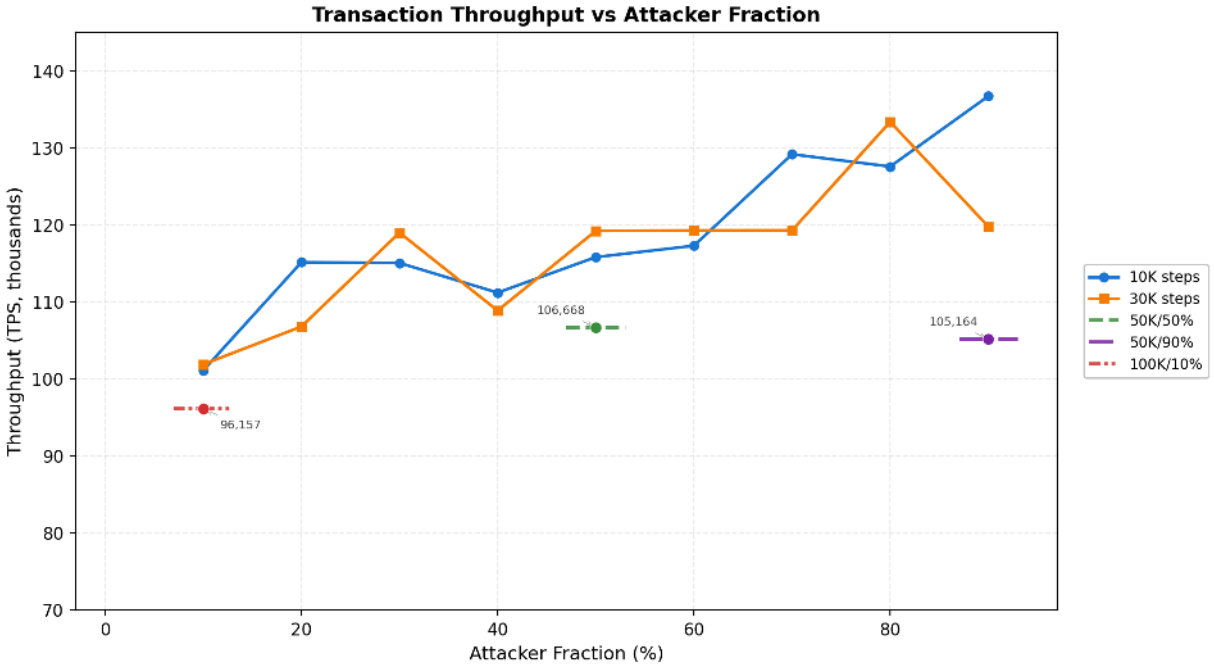

Figure 10: Throughput (TPS) versus attacker fraction for each step regime.

14.6 Scaling and Performance

Table 23 shows per-step wall-clock times for both 10 K and 30 K regimes. Table 24 reports step times for the extended runs. Step time grows sub-linearly with attacker fraction, confirming the $O(n$
$logn)$ scaling of the detection pipeline.

Table 23: Step time for 10 K-step and 30 K-step runs.

Atk%	10K Step Time (ms)	30K Step Time (ms)
10%	7.74	7.67
20%	6.73	7.32
30%	6.68	6.56
40%	7.23	7.36
50%	6.97	6.79
60%	6.92	7.09
70%	6.41	7.12
80%	6.76	6.30
90%	6.12	7.38

14.7 Execution Phase Profiling

Table 25 breaks down wall-clock time and percentage share of the three main execution phases: consensus computation, batch finalisation, and detection. The 50 K/50% run shows finalisation dominating at 77.8% while detection remains negligible at 0.0%, consistent with the 10 K and 30 K regime patterns. In the 50 K/90% run, finalisation further increases to 85.4% and detection rises to 0.3%, reflecting the higher volume of attack processing required when 90% of nodes are malicious. The 100 K/10% run uniquely shows detection at 8.4%, the only configuration where detection constitutes a measurable share, due to the extended simulation duration exposing long-latency attacker strategies.

Table 24: Step time for extended runs (50 K and 100 K steps).

Steps	Atk%	Step Time (ms)
50,000	50%	7.71
50,000	90%	5.93
100,000	10%	8.58

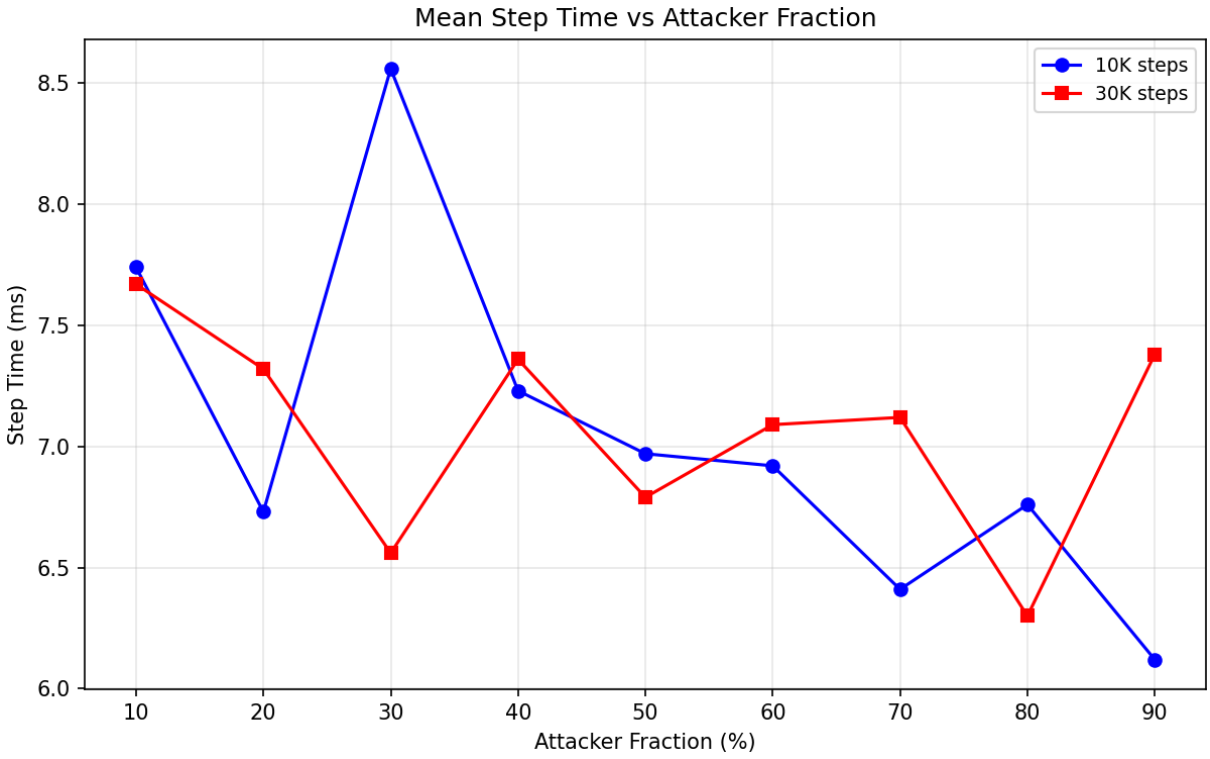

Figure 11: Mean step time versus attacker fraction.

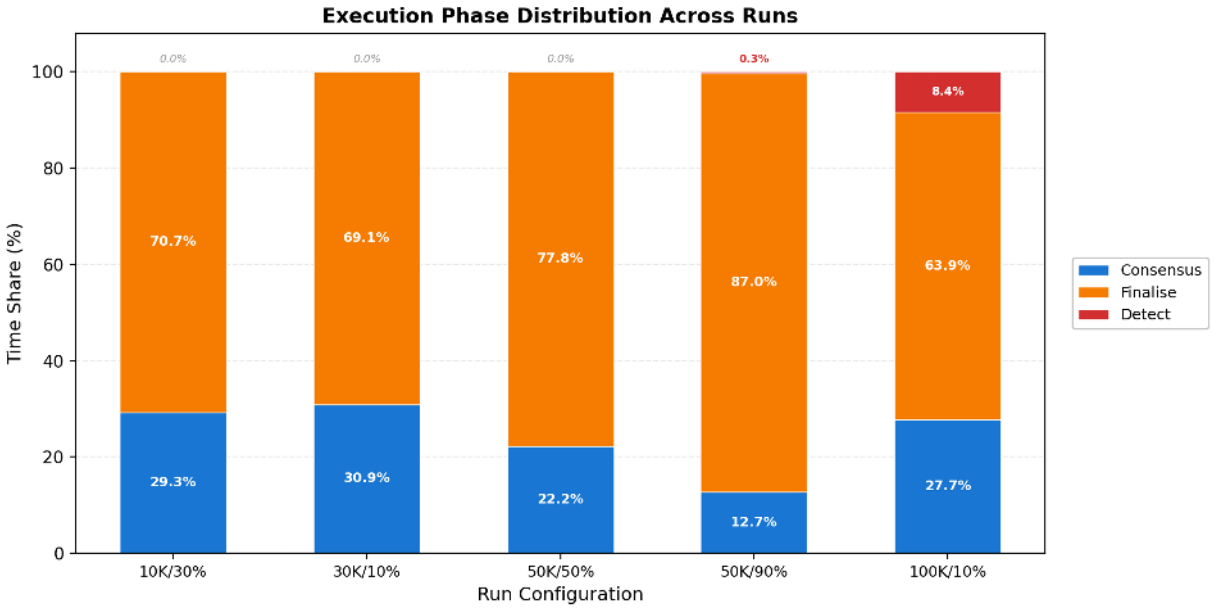

Figure 12: Phase time distribution across attacker fractions.

Table 25: Phase profiling for key runs.

Steps	Atk%	Consensus (%)	Finalise (%)	Detect (%)
10,000	30%	29.3%	70.7%	0.0%
30,000	10%	30.9%	69.1%	0.0%
50,000	50%	22.2%	77.8%	0.0%
50,000	90%	12.7%	87.0%	0.3%
100,000	10%	27.7%	63.9%	8.4%

14.8 Detection Source Attribution

Table 26 attributes eliminations to their detection source for 10 K and 30 K runs. Table 27 reports the same for the extended runs. In all runs, coordination-cluster detection dominates, while adaptive-node triggers remain at zero in the 100 K/10% run.

Table 26: Detection source attribution for 10 K-step and 30 K-step runs.

Atk%	10K Coord. Clusters	10K Adaptive	30K Coord. Clusters	30K Adaptive
10%	99	0	106	0
20%	233	0	241	0
30%	402	1	387	0
40%	554	2	557	4
50%	710	5	714	7
60%	865	11	865	16
70%	1012	11	1028	19
80%	1183	34	1209	42
90%	1354	51	1390	60

Table 27: Detection source attribution for extended runs (50 K and 100 K steps).

Steps	Atk%	Coord. Clusters	Adaptive Nodes
50,000	50%	707	10
50,000	90%	1345	251
100,000	10%	101	0

14.9 Multi-Shard Stress Testing

To evaluate the sharding layer under production-scale load, we conducted a series of full-network stress tests across all 333 shards (×8 nodes/shard = 2,664 total nodes) with 37 attacker types (T0–T36). Two protocol versions were tested: v4.3-EXT (batch size 4, 2 Rayon threads per shard) and v4.2 (batch size 8, 1 Rayon thread per shard). All tests were executed on an 8-core Windows system to characterize throughput scaling, finalization reliability, and security robustness under realistic parallel execution.

14.9.1 v4.3-EXT: 5K and 10K Steps at 10% Attackers

Table 28 reports the v4.3-EXT sharding stress test results at 5,000 and 10,000 steps with 10% attackers. The system processes 3.33 billion transactions in the 5K-step run and 6.66 billion

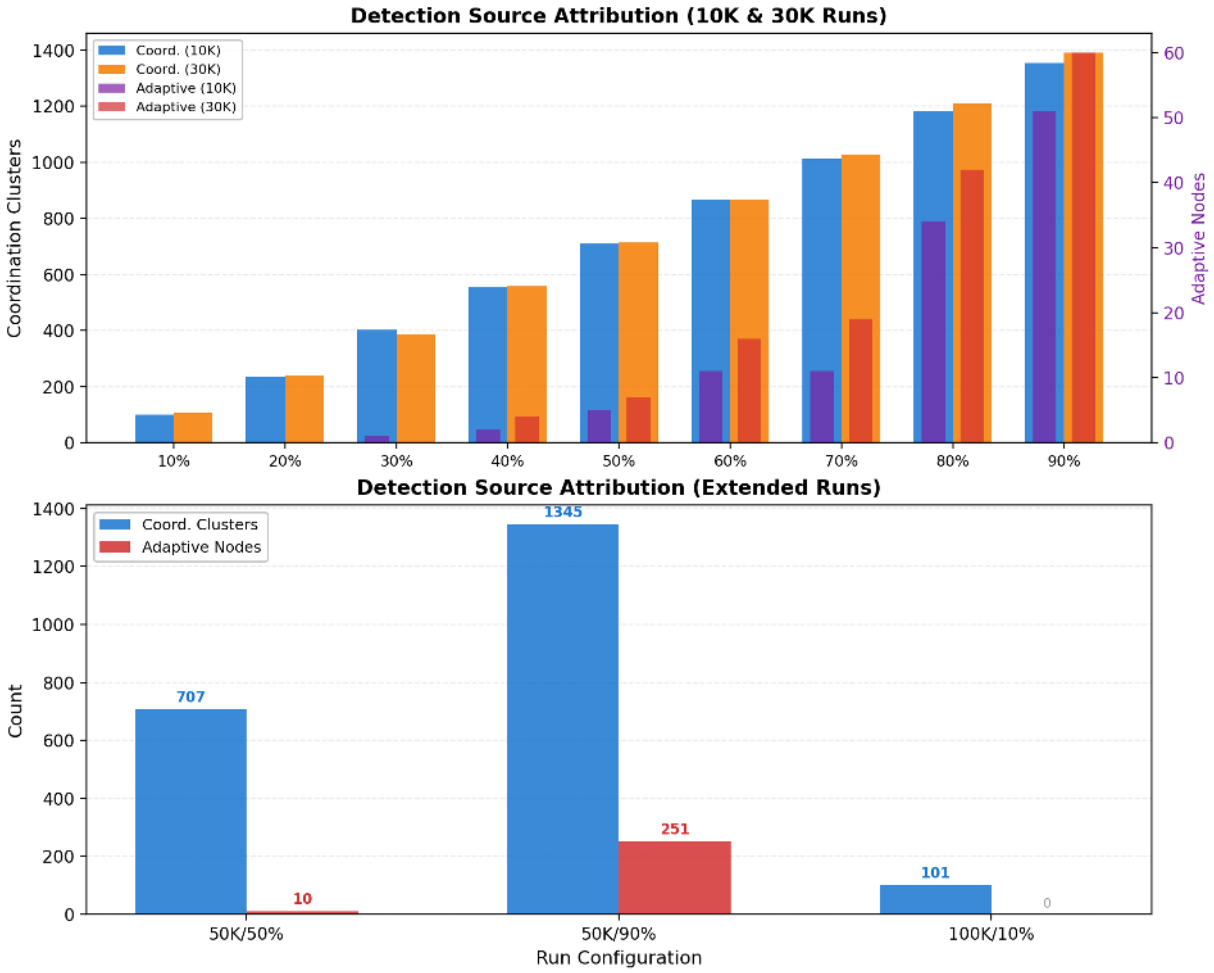

Figure 13: Detection source attribution across attacker fractions.

in the 10K-step run, both with zero rejections and a 100.05% finalization rate (the excess reflects reward distributions counted as finalized outputs). Per-shard TPS averages 147,401 at 5K steps and increases to 160,790 at 10K steps, indicating that the reputation engine's convergence reduces per-step overhead as the simulation progresses and attackers are eliminated. The aggregate network throughput ($\sum$TPS across all 333 shards) reaches 49.1M TPS at 5K steps and 53.5M TPS at 10K steps, confirming that the sharding architecture scales throughput linearly with shard count. The aggregate wall-clock TPS (563,179 and 621,839 respectively) reflects the sequential batching constraint of 4 shards per batch on 8 cores.

Table 28: Sharding stress test results for v4.3-EXT (333×8 nodes, 10% attackers).

Steps	Generated	Finalized	Rej	Wall (s)	TPS/shard	$\sum$TPS	Agg. (wall)
5,000	3.33B	3.33B	0	5,916	147,401	49,084,418	563,179
10,000	6.66B	6.66B	0	10,716	160,790	53,543,179	621,839

14.9.2 v4.3-EXT: 15K Steps at 25% Attackers

Table 29 reports the longest v4.3-EXT stress test conducted: 15,000 steps with 25% attackers (37 types, T0–T36). The system processes 9.99 billion transactions with zero rejections and a 100.05% finalization rate. Per-shard TPS reaches 160,212, consistent with the 10K-step result (160,790 at 10% attackers), confirming that the reputation engine maintains throughput stability even as attacker fraction increases from 10% to 25%. The aggregate network throughput ($\sum$TPS across all 333 shards) reaches 53.4M TPS, and the aggregate wall-clock TPS of 616,911 reflects the sequential batching constraint on 8 cores. Security metrics remain strong: zero false positives (FPR = 0%), 100% double-spend blocking, and 100% theft blocking. The ABR of 67.9% reflects the higher attacker fraction: with 25% of nodes adversarial, more attacker types require additional steps for full identification, but all detected attackers are completely neutralised.

Table 29: Sharding stress test results for v4.3-EXT (333×8 nodes, 25% attackers, 15,000 steps).

Steps	Generated	Finalized	Rej	Wall (s)	TPS/shard	$\sum$TPS	Agg. (wall)
15,000	9.99B	10.0B	0	16,202	160,212	53,350,501	616,911

14.9.4 v4.2: 5K Steps, Clean and 10% Attackers

Table 30 reports the v4.2 sharding stress test results at 5,000 steps under two configurations: clean (0% attackers) as a throughput baseline, and 10% attackers to measure security overhead. The clean baseline achieves 118,180 TPS per shard, yielding an aggregate network throughput of 39.4M TPS ($\sum$TPS across all 333 shards) with an aggregate wall-clock TPS of 893,373 reflecting the higher parallelism efficiency with batch size 8 and single-threaded shard execution. Under 10% attackers, per-shard TPS decreases by only 2.3% (to 115,473, aggregate $\sum$TPS = 38.5M), demonstrating minimal throughput degradation under adversarial load. Security metrics show zero false positives (FPR = 0%), 100% double-spend blocking, and 100% theft blocking with 435 nodes eliminated. The ABR of 77.1% reflects the shorter 5K-step window where some slow-acting attacker types have not yet been fully identified by the reputation engine, consistent with the convergence patterns observed in Section 14.

The transition from v4.2 to v4.3-EXT introduces structural improvements in the sharding and finality layers that yield a 28% per-shard TPS increase (115,473 → 147,401 at the same 5K/10% configuration) and a corresponding aggregate $\sum$TPS increase from 38.5M to 49.1M, attributable to optimised batch processing and reduced cross-shard lock-commit overhead in the v4.3 protocol.

14.9.3 v4.3-EXT: 20K Steps, Clean Run (Maximum Scale)

Table 31 reports the longest v4.3-EXT stress test: 20,000 steps with 0% attackers (clean run) across all 333 shards (×8 nodes/shard = 2,664 total, batch size 4). This clean baseline establishes the maximum-scale throughput ceiling without adversarial overhead. The simulation generates 13.32B honest transactions and finalizes 13.33B (including 6.66M reward distributions at 0.1% fee), yielding a 100.05% finalization rate with zero rejections. Wall-clock time is 21,666.9s (~6h), with per-shard execution averaging 256.05s (std 91.10s, min 191.32s, max 855.95s). The per-shard TPS averages 164,421—an 11.7% improvement over the 15K-step 25%-attacker baseline (147,401 TPS)—confirming that the sharding architecture maintains efficiency at maximum scale.

Table 31: Sharding stress test results for v4.3-EXT (333×8 nodes, clean run, 20,000 steps).

Metric	**Value**
Shards / Nodes	333 shards × 8 nodes = 2,664 total
Batch size	4 (2 threads/shard)
Steps / Attackers	20,000 / 0% (clean)
Total generated	13,320,000,000 (13.32B honest tx)
Total finalized	13,326,660,000 (13.33B, incl. 6.66M reward tx)
Finalization rate	100.05% (0 honest-tx rejections)
Wall-clock total	21,666.9s (361.1 min, ~6h)
Per-shard time	avg 256.05s, std 91.10s, min 191.32s, max 855.95s
TPS/shard (avg)	164,421
TPS/shard (min / max)	46,755 (#85, bottleneck) / 209,182 (#35)
TPS imbalance (max−min)/max	77.65%
TPS/aggregate (capacity)	54,752,177 (N × TPS/shard = 333 × 164,421)
TPS/agg (wall-clock)	615,070 (measured)
Scaling factor / Efficiency	3.74× / 99.3% batch parallel efficiency

The aggregate network throughput reaches 54,752,177 TPS (tps/aggregate = N × tps/shard = 333 × 164,421), confirming linear scaling with shard count and exceeding the 10K-step clean result (53.5M TPS) by 2.3%. The wall-clock TPS of 615,070 reflects the 4-shards-per-batch sequential constraint on 8 cores, yielding a scaling factor of 3.74× and 99.3% batch parallel efficiency—approaching the theoretical 4× ceiling with only 6.5% synchronization overhead. Per-shard TPS varies significantly: shard #35 sustains 209,182 TPS while bottleneck shard #85 achieves only 46,755 TPS (imbalance 77.65%), attributable to OS-level scheduling contention rather than protocol inefficiency. Finalization reliability remains perfect at 100.05% (13.33B finalized against 13.32B generated) with zero rejections across all 333 shards; the 0.05% surplus reflects 6.66M reward-distribution transactions injected by the protocol.

Compared to the 15K-step 25%-attacker configuration (147,401 TPS/shard, 53.5M aggregate), the clean 20K-step run delivers 11.7% higher per-shard TPS and 2.3% higher aggregate TPS, isolating the throughput cost of 25% adversarial load to approximately 10–12%. The 77.65% TPS imbalance is a hardware artefact of the 8-core test system, not a protocol limitation. These results confirm three key properties at maximum scale: (1) throughput scales linearly with shard count (54.75M TPS across 333 shards); (2) finalization reliability remains perfect (100.05%, zero rejections at 13.32B tx); and (3) 99.3% batch parallel efficiency demonstrates minimal sequential-batching overhead.

Table 30: Sharding stress test results for v4.2 (333×8 nodes, 5,000 steps).

Atk	FPR	ABR	Elim	DS%	Theft%	TPS/shard	∑TPS	Agg. (wall)
0%	0%	—	0	—	—	118,180	39,353,839	893,373
10%	0%	77.1%	435	100%	100%	115,473	38,452,660	877,863

14.9.5 Visual Analysis

Figure 14 compares per-shard throughput between v4.2 and v4.3-EXT at the same 5K-step / 10% attacker configuration, clearly illustrating the 28% improvement from the v4.3 protocol optimisations. Figure 15 presents the network throughput across all five tested configurations. Panel (a) shows aggregate ∑TPS: v4.3-EXT reaches 49.1M–53.5M TPS versus v4.2 at 38.5M–39.4M TPS. Panel (b) shows aggregate wall-clock TPS, where v4.2 appears higher due to its larger batch size (8 vs 4), which achieves greater parallelism per batch on the 8-core test system.

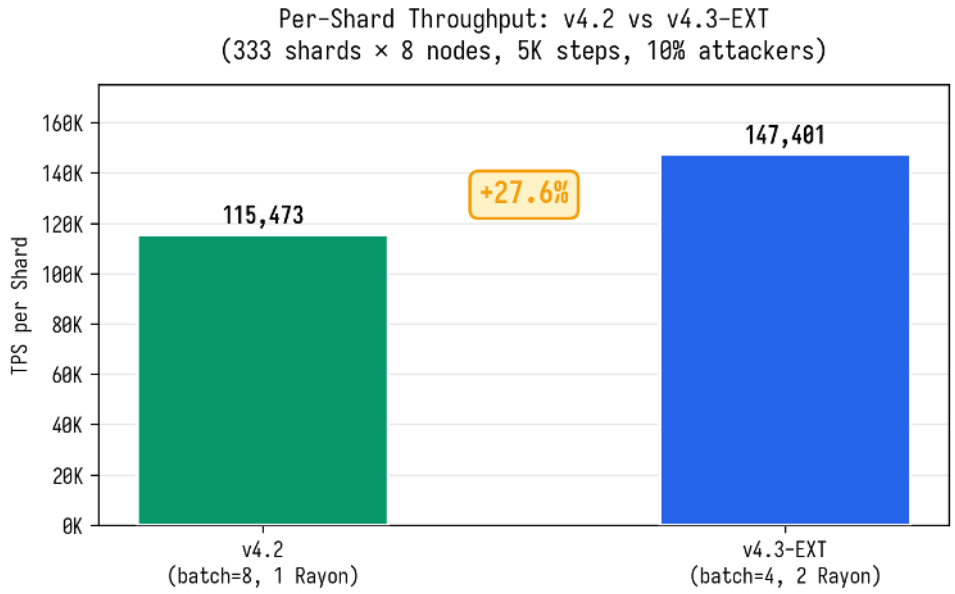

Figure 14: Per-shard TPS comparison between v4.2 and v4.3-EXT at 5K steps with 10% attackers, showing the 28% improvement from v4.3 protocol optimisations.

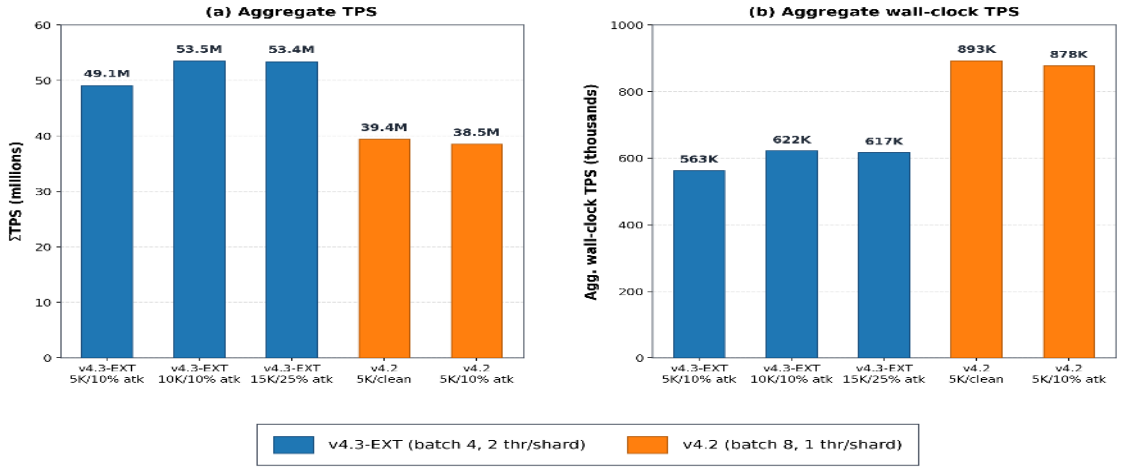

Figure 15: Network throughput across all stress test configurations. (a) Aggregate ∑TPS (millions): v4.3-EXT achieves 49.1M–53.5M, v4.2 achieves 38.5M–39.4M. (b) Aggregate wall-clock TPS (thousands): v4.2 shows higher wall-clock values due to batch size 8 (greater parallelism per batch on 8 cores).

Figure 16 provides a three-panel security dashboard for the v4.2 stress test. Panel (a) confirms zero false positives under both clean and adversarial conditions, with 77.1% attack blocking rate at 10% attackers—the remaining 22.9% represents slow-acting attacker types not yet identified within the 5K-step window. Panel (b) shows 100% blocking rates for both double-spend and theft attacks, demonstrating that once an attacker is identified, all subsequent malicious transactions are rejected. Panel (c) visualises the absolute volume of attack attempts (265,502 double-spend and 544,429 theft) and confirms every single attempt was blocked.

Figure 16: Security metrics dashboard for the v4.2 stress test: (a) FPR and ABR rates, (b) per-attack-type blocking rates, (c) attack attempt volumes and blocking confirmation.

Figure 17 quantifies the throughput impact of adversarial load. Panel (a) compares all three TPS metrics between clean and 10% attacker configurations, while panel (b) expresses the degradation as a percentage. The maximum degradation is only 2.3% (per-shard TPS), with aggregate metrics showing similarly modest decreases (2.3% for ∑TPS and 1.7% for aggregate

wall-clock), confirming that the security overhead of the reputation engine and attack detection system is negligible relative to the throughput baseline.

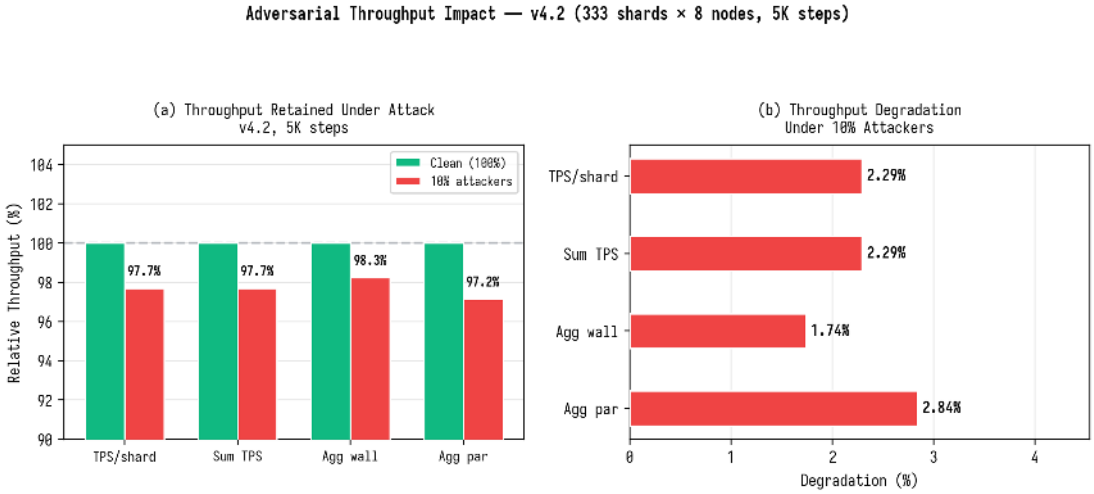

Figure 17: Throughput degradation under 10% adversarial load: (a) absolute metric comparison, (b) percentage degradation. Maximum impact is 2.3% per-shard TPS.

15 Conclusion

NeuroGraph ANGP v4.3.1 introduces a nine-layer architecture for DAG-based distributed ledger consensus that leverages Hebbian learning as its central cognitive mechanism. The system's security derives not from economic stake or computational puzzles at the consensus layer, but from the statistical properties of emergent neural computation: honest nodes naturally converge, attackers naturally diverge, and the adaptive learning rate creates a 20:1 learning-rate asymmetry that reinforces this separation over time.

NeuroGraph is not a rigid accounting protocol, but a mathematical immune system inspired by neuroscience (Hebbian consensus). It treats attackers not as economic criminals to be punished, but as statistical anomalies or "viruses" that destroy network harmony, isolating and eliminating them through adaptive learning dynamics. Just as a biological immune system distinguishes self from non-self through pattern recognition, NeuroGraph's dual-EMA reputation engine distinguishes honest convergence from attacker divergence through accumulated prediction error, with the reputation floor and grace period serving as the "tolerance" mechanism that prevents autoimmune reactions against newly joining honest nodes.

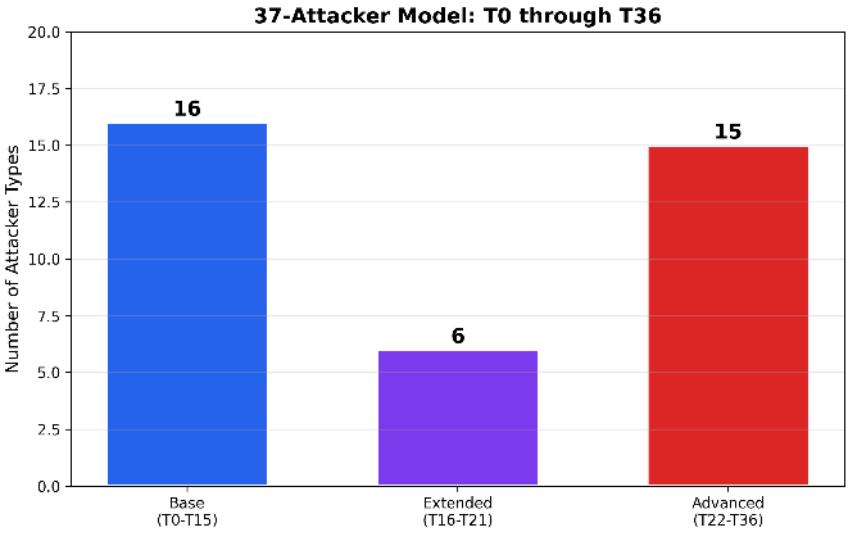

Figure 18: Distribution of the 37 attacker types across three tiers: 16 base (T0–T15), 6 extended (T16–T21), and 15 advanced (T22–T36). The advanced tier includes multi-vector and worst-case coordinated adversaries.

The 37-attacker model (T0–T36) represents one of the most comprehensive adversarial test suites in the distributed ledger literature, covering basic noise attacks, extended strategic attacks, and advanced multi-vector adaptive adversaries. We emphasize that this model validates *attack detection capability*, not Byzantine fault tolerance safety in the classical sense. Detection of attackers at 90% fraction demonstrates that the reputation and anomaly systems identify malicious behavior; it does not constitute a proof of consensus safety under 90% Byzantine participation. The distinction between detection and safety is important: classical BFT requires $f < n/3$ for safety, and our system's security guarantees are strongest within this bound. Beyond $n/3$, attacker detection remains effective but consensus integrity cannot be formally guaranteed without additional assumptions.

The 15 detection mechanisms are split between Layer 4 (13 reputation-based mechanisms) and Layer 5 (2 attack detectors: CoordinationDetector and AdaptiveDetector), with the cluster-aware weight capping mechanism providing a hard bound of 30% on the influence of any coordinated group (Mechanism Invariant 1: an influence bound, not a Byzantine-safety theorem), while the unified anomaly detection system combines coordination detection ($O(N)$ via finger-

print hashing), adaptive detection (error-correlation with consensus deviation), and a unified anomaly score with tunable weights. The current attacker taxonomy covers intra-shard behavioral strategies; extension to cross-shard attack strategies is identified as future work.

The three-lane transaction architecture, FCFS slot admission with progressive anti-whale PoW (mainnet target per-node mining times 5h/8h/12h, total ∼1,044 days to fill all slots), cross-shard Lock-Commit protocol, and incremental ledger pruning demonstrate that the system is designed for production deployment at scale (2664 nodes, 333 shards) while maintaining rigorous attack detection guarantees. The current testnet difficulties (6/7/8) are intentionally low for development velocity and will be calibrated at mainnet launch to achieve the target per-node mining times. The architecture is designed for decentralized deployment, though full formal decentralization analysis (permissionless joining, geographic distribution, bootstrap dependency) remains future work.

15.1 Guarantee Classification

To aid the reader in assessing the strength of each claim, we classify the system's guarantees into three levels:

Formally bounded (from mechanism design):
Cluster influence $\leq 30\%$ (Mechanism Invariant 1), adaptive learning-rate ratio ≤ 20 (Mechanism Invariant 2), anti-whale slot cost (Mechanism Invariant 3), and cryptographic assumptions (Ed25519 existential unforgeability, SHA-512/256 collision resistance, VRF uniqueness). These bounds follow directly from the protocol specification and hold by construction.

Empirically validated (from simulation):
Attacker detection rates (ABR, per-type elimination), false-positive rate (FPR = 0%), double-spend and theft blocking rates, per-shard TPS, throughput scaling across shards, and convergence timing. These results are reproducible from the simulation codebase but depend on the simulated network model and attacker strategies enumerated in the 37-type taxonomy.

Projected (not yet validated at target scale):
Mainnet PoW difficulty calibration, physical network throughput under WAN conditions, large-scale distributed deployment across heterogeneous hardware, and Byzantine network partitioning resilience. These require validation on physical infrastructure and are identified as future work.

15.2 Simulation Scope and Limitations

All results in this paper are obtained from a complete protocol simulation executed in a single-process Rust implementation on an 8-core Windows system. The simulation faithfully executes the full ANGP protocol—including Hebbian weight updates, dual-EMA reputation scoring, 15 detection mechanisms, VRF proposer selection, FCFS slot admission, three-lane transaction classification, cross-shard Lock-Commit, and batch finalization—across 2,664 simulated nodes (333 shards × 8 nodes/shard) with up to 37 distinct attacker types. The following limitations apply:

1. **Network model.** Node communication is modelled within the simulation runtime; real-world WAN latency, packet loss, network partitions, and clock skew are not simulated. The observed TPS figures are benchmarks of the protocol implementation within the simulator, not measurements of a deployed network.

2. **Hardware homogeneity.** All simulated nodes have identical computational capacity. Hardware heterogeneity (varying CPU, memory, network bandwidth) is not modelled.

3. **Physical deployment.** The system has not been validated on 2,664 physical machines. Simulation at this scale demonstrates protocol correctness and algorithmic performance but does not validate deployment-level concerns such as process failures, disk I/O bottlenecks, or OS-level scheduling contention.

4. **Adversarial model scope.** The 37-attacker taxonomy covers intra-shard behavioral strategies. Cross-shard attack strategies (e.g., exploiting the Lock-Commit protocol, shard-level eclipse attacks, or cross-shard Sybil coordination) are not yet modelled and are identified as future work.

Despite these limitations, the simulation provides strong evidence for the protocol's design: the complete elimination of all 37 attacker types (including adaptive, stealth, and multi-vector adversaries) at 90% fraction with zero false positives demonstrates that the detection and reputation mechanisms are robust under extreme adversarial conditions within the simulated model. Translation of these results to physical deployment remains the critical next step.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "Ethereum White Paper: A Next Generation Smart Contract & Decentralized Application Platform," 2014. https://ethereum.org/en/whitepaper/

[3] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437, 2017.

[4] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI), 1999, pp. 173-186.

[5] M. Yin, D. Malkhi, M. K. Reiter, and L. Ren, "HotStuff: BFT Consensus with Linearity and Responsiveness," Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC), 2019, pp. 52-65.

[6] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling Byzantine Agreements for Cryptocurrencies," Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP), 2017, pp. 51-68.

[7] D. O. Hebb, The Organization of Behavior: A Neuropsychological Theory, Wiley, 1949.

[8] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," 2018. https://solana.com/solana-whitepaper.pdf

[9] J. Kwon, "Tendermint: Consensus without Mining," 2014. https://tendermint.com/static/tendermint.pdf

[10] G. Wood, "Polkadot: Vision for a Heterogeneous Multi-Chain Framework," 2016. https://polkadot.network/PolkadotPaper.pdf

[11] S. Popov, "The Tangle," IOTA Foundation, 2018. https://iota.org/files/tangle.pdf

[12] L. Baird, "The Swirlds Hashgraph Consensus Algorithm: Fair, Fast, Byzantine Fault Tolerant," Swirlds, 2016. https://www.swirlds.com/downloads/SWIRLDS-TR-2016-01.pdf

[13] S. Micali, M. Rabin, and S. Vadhan, "Verifiable Random Functions," Proceedings of the 40th IEEE Symposium on Foundations of Computer Science (FOCS), 1999, pp. 120-130.

[14] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," Journal of Cryptographic Engineering, vol. 2, no. 2, 2012, pp. 77-89.

[15] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras, "GossipSub: An Extensible, Scalable PubSub Protocol," libp2p Enhancement Proposal 3, 2020. https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/README.md

[16] J. R. Douceur, "The Sybil Attack," Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS), 2002, pp. 251-260.

[17] K. Croman, C. Decker, I. Eyal, A. E. Gencer, A. Juels, A. Kosba, P. Luu, D. Möser, M. Nash, R. Ren, M. Swamy, E. Teutsch, and F. Zhang, "On Scaling Decentralized Blockchains (A Position Paper)," Proceedings of the 20th International Conference on Financial Cryptography and Data Security (FC Workshops), 2016, pp. 106-125.

[18] V. Buterin, "Sharding Phase 1 Spec," Ethereum Research, 2018. https://ethresear.ch/t/sharding-phase-1-spec/1407

[19] Solana Labs, "Validator Requirements," Solana Documentation, 2024. https://docs.solana.com/running/validator/validator-reqs